



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INFORMATION SYSTEMS

ÚSTAV INFORMAČNÍCH SYSTÉMŮ

HEADLESS CMS IN CLOUD

HEADLESS CMS IN CLOUD

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

FILIP HOLČÍK

SUPERVISOR

VEDOUCÍ PRÁCE

Ing. VLADIMÍR BARTÍK, Ph.D.

BRNO 2025

Bachelor's Thesis Assignment



166024

Institut: Department of Information Systems (DIFS)
Student: **Holčík Filip**
Programme: Information Technology
Title: **Headless CMS in Cloud**
Category: Web
Academic year: 2024/25

Assignment:

1. Study the existing solutions in the field of CMS, evaluate the shortcomings of the CMS used today. Learn about web development technologies.
2. Analyze the requirements for a headless CMS, which, in addition to basic functionality, can be adapted to the use of various cloud services, it will be possible to run it offline, and it will enable serialization/deserialization of the configuration.
3. Design a CMS meeting the requirements from point 2.
4. Implement the proposed CMS and test its functionality on a suitable data set.
5. Evaluate the achieved results and other options for continuing this project.

Literature:

- Tomlinson, T.: Beginning Backdrop CMS. Apress, 2016. ISBN 978-1484219690.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Bartík Vladimír, Ing., Ph.D.**
Head of Department: Kolář Dušan, doc. Dr. Ing.
Beginning of work: 1.11.2024
Submission deadline: 14.5.2025
Approval date: 22.10.2024

Abstract

When there is a need for a CMS during the development of an information system, its choice is crucial, as the developed information system will inherit its features and limitations. After the advent of cloud computing, there was no shift in CMS development to leverage cloud architecture fully. Most current CMS solutions fixate on the platform they run on, usually one. In the world of the cloud, there is no single platform; instead, there are many different ones with shared similarities. This work aims to propose and develop a headless CMS solution that is adaptable to run on any platform, with easy migration between them, a withdrawable configuration, and the ability to work offline.

Abstrakt

V prípade potreby CMS počas vývoja informačného systému musia vývojári urobiť zásadnú voľbu, nakoľko táto voľba vplýva na funkcionality a limitácie vyvíjaného systému. Po príchode cloudov nebola zaznamenaná žiadna zásadná zmena vo vývoji CMS, ktorá by umožnila využitie plného potenciálu architektúry cloudu. Väčšina existujúcich CMS riešení je spustiteľná na obmedzenom počte platforiem, zvyčajne práve na jednu. Lenže cloud nie je jedna platforma, ale viacero platforiem, ktoré sa navzájom podobajú. Cieľom tejto práce je navrhnúť a vyvinúť headless CMS riešenie, ktoré je možné adaptovať na ktorúkoľvek platformu, s jednoduchou migráciou medzi platformami, exportovateľnou konfiguráciou a schopnosťou fungovať aj bez pripojenia na internet.

Keywords

CMS, headless, cloud, platform-agnostic

Klíčová slova

CMS, headless, cloud, platformovo nezávislé

Reference

HOLČÍK, Filip. *Headless CMS in Cloud*. Brno, 2025. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Vladimír Bartík, Ph.D.

Headless CMS in Cloud

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of doctor Vladimír Bartík. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis.

.....
Filip Holčík
April 3, 2025

Acknowledgements

I want to express my gratitude to Ing. Vladimír Bartík Ph.D. for his support and understanding throughout this project.

Contents

1	Introduction	4
2	CMS	5
2.1	Types of CMS	6
2.1.1	Classification by architecture	6
2.1.2	Classification by purpose	7
2.1.3	Classification by platform	7
2.1.4	Classification by availability	9
2.1.5	Classification by target audience	9
2.2	Cloud based systems	9
2.2.1	Database and storage	10
2.2.2	Compute	11
2.3	Problem with CMS on cloud	12
3	Requirements	15
3.1	Managing database content	16
3.2	Managing file storage	17
3.3	Component API	18
3.4	Summary of requirements	19
4	Proposed solution	20
4.1	Ensuring portability	20
4.1.1	@genoacms/cloudabstraction	21
4.1.2	Adapters	23
4.2	Scalability	24
4.3	Selecting tools	26
4.3.1	Programming language	26
4.3.2	Framework	31
4.3.3	User interface	32
4.3.4	Data scheming and validation	32
4.4	General architecture	33
4.4.1	Layered architecture	35
4.4.2	Package structure	36
4.5	Configuration	37
4.5.1	Providers API	40
4.5.2	Authentication	42
4.5.3	Authorization	42
4.5.4	Database	42

4.5.5	Deployment	42
4.5.6	Storage	43
4.6	Server side	43
4.6.1	Authentication and authorization	44
4.6.2	Routing	45
4.7	Client side	46
4.7.1	User interface	46
4.7.2	Component API	47
4.8	CLI utility	52
4.8.1	Deployment	53
4.9	Offline capabilities	54
4.9.1	Service emulators	54
4.9.2	Special adapters	55
5	Implementation and testing	56
5.1	Major challenges	57
5.1.1	Page manager	57
5.1.2	Cloud Run SvelteKit adapter	58
5.1.3	Build	59
5.1.4	Loading dynamic configuration	59
5.2	Testing	60
5.2.1	Real world usage	60
6	Conclusion	63
	Bibliography	64
A	The future	66
A.1	Coding components	66
A.2	Activity tracking	67
A.3	Artificial intelligence service	67
A.4	Onboarding developers	67

List of Figures

2.1	Diagram clarifying terms	5
2.2	Architecture shift from monolithic to service-oriented (cloud)	10
2.3	Example structure of Strapi alongside with order system	13
2.4	Cloud SQL and Firestore budget comparison	14
4.1	Simplified comparison of programming languages	27
4.2	Most used programming languages in 2024 according to Statista https://www.statista.com/statistics/793628/worldwide-developer-survey-most-used-languages/	28
4.3	General architecture diagram	33
4.4	Layered architecture diagram	35
4.5	Type declaration of configuration	38
4.6	Example of storage config with multiple providers	41
4.7	Overview of CMS API Functions	44
4.8	Route structure	45
4.9	User interface screenshot	46
4.10	ComponentEntry structure proposal	48
4.11	ComponentDefinition structure proposal	49
4.12	Component structure proposal	49
4.13	Simplified PageEntry structure proposal	51
4.14	ReadablePageNode structure proposal	52
4.15	Deployment visualization	53
4.16	Running GenoaCMS offline using service emulators	54
4.17	Running GenoaCMS offline using special services	55
5.1	Cost analysis of a website using GenoaCMS	61
5.2	Cost analysis of a website using GenoaCMS	61

Chapter 1

Introduction

When selecting a content management system (CMS), the developer has to consider many factors, as the choice of CMS significantly impacts the future capabilities of the developed system. This makes that choice on par with other important aspects like programming language or target platform.

The current state of CMS solutions is that there is a plethora of existing solutions, each with its feature set and technological prerequisites. Thus, building a system on a specific technology stack can limit the choice of CMS. Having CMS and its technology stack bound leads to a disparity in functionalities across platforms. Specific platforms and technologies lack CMS solutions altogether. Developers address that by having multiple platforms for their system or creating ad-hoc CMS. This practice of creating ad-hoc CMS has been a target of ridicule within the development community, and rightfully so. The cycle of reinventing the wheel for each platform does not address the original problem - lack of **platform-agnostic CMS** solutions.

The problem with current CMS solutions goes deeper than just the lack of platform independence. Scaling and cost efficiency when using CMS on cloud-based systems can be real issues. Also, a widespread false belief is that one needs an internet connection to use a CMS.

Many existing CMS solutions do not even consider the possibility of switching platforms, and withdrawing configuration programmatically is impossible for many of them. It seems like the industry is more focused on adding new features than addressing essential, non-functional requirements.

The goal is to create a proof-of-concept CMS that shows it is possible to build a system with essential qualities overlooked by the industry.

This work begins by exploring CMS solutions and their possible use cases, dually exploring cloud-based systems and problems that arise when using CMS on the cloud. This will help us define the requirements for the new CMS solution, from which we will derive the architecture and implementation details. Finally, we will evaluate the solution and discuss its limitations and possible future improvements.

Chapter 2

CMS

This thesis will often mention the existing system and CMS as two counterparts. The developed system is an information system composed of a back-end and applications (the existing system) and a newly integrated CMS. Back-end and applications are the main focus of the development, and the CMS is the tool for managing the system's content. They have their differences, and the developer's goal is to make them work together seamlessly, either by customizing the system or CMS.

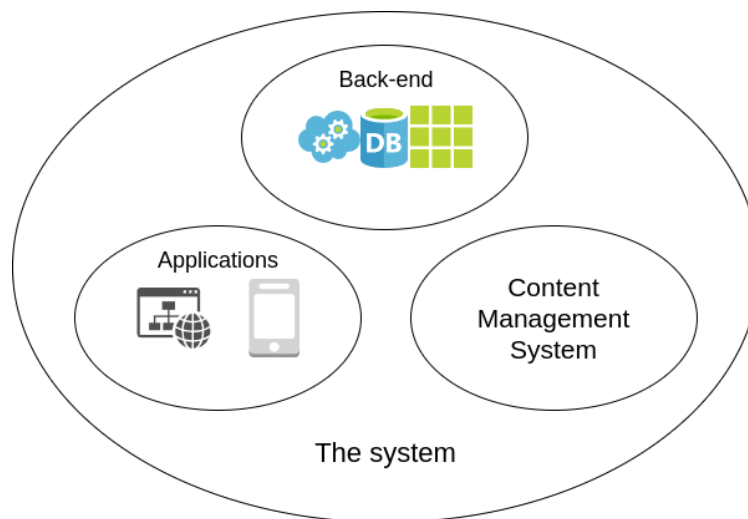


Figure 2.1: Diagram clarifying terms

A content management system is a mechanism that allows users of information systems to manage the system's content without needing to rebuild the system's code with new data or know how to interact with the database. A more general definition refers to it as a software package that provides some level of automation for the tasks required to manage content effectively[1]. By this definition, the system itself could be considered a CMS, but in practice, the system developer cannot afford to implement CRUD operations for every entity in the system.

Consider an order system for a store selling paints. The system has to allow making purchases and managing stock, but among other things, the system features a blog. The blog is not the system's main feature, but the store needs it. The developer has to decide whether to implement the blog as part of the system or use an existing solution. The first

option would require the developer to implement CRUD operations for blog posts, post images, and others. This does not seem like much work, but customers' expectations are often high, and implementing a fancy interface for managing blog posts raises the project's cost. Thus, the developers are pushed to use an existing program that allows managing the system's content - CMS.

Usually, CMS solutions store data in a specific database and run in a specific environment. Given that there is a zillion of databases and environments, the reality is that the database and environment do not match. This means two back-ends, two deployments, and often two invoices at the end of the day. Let us pretend this is not a problem and that the order system's developers use a CMS with a different tech stack. Consider that the system is in Java, and the CMS is in PHP. The Java developers do not have any problem integrating the CMS according to documentation, but the client insists on having a super fast responding blog, so the team decided to implement incremental static regeneration; every time the user changes the content of the blog, the system needs to prerender the blog statically. However, there is a problem: the CMS stores data in a database other than the rest of the system. The CMS can catch the event of changing blog articles, but not their Java system; the Java developers can implement a polling mechanism to probe for new changes, but since the client wants instant updates, this is not an option. This means that Java developers have to write PHP code to propagate the article change to their web server so that it can regenerate the static content. This example seems far-fetched, but it highlights the shortcomings of current CMS solutions:

- CMS is not part of the system; it is a separate system
- CMS is written in a different language than the system
- CMS is using another database as the system
- CMS is not running in the same environment as the system
- One team member has to know the implementation details of the CMS

This is a huge price for not implementing CRUD operations for blog posts, but for some systems, it's easier to use CMS, especially when there are more types of content to manage.

2.1 Types of CMS

To better understand CMS solutions, we will divide them by key factors. It is important to note that there is no official classification of CMS solutions; this information is purely based on the author's experience.

2.1.1 Classification by architecture

This factor impacts the usage mainly. It defines what CMS considers managed content. Content can be data like text, images, even UI, or, to some extent, even script.

- **Traditional CMS** - this type provides front-end to back-end solutions. User can define individual pages with their UI, generic pages with variable data, and even more complex mechanisms like forms. With this much capability, the CMS becomes complex and introduces its challenges and limitations. When the CMS manages UI,

it must have some tool for creating it in a user-friendly way and then interpreting it in a computer-readable way; this means some parser has to be present, and the parser must parse the UI (internally as a nested structure) to a format that can be displayed on user's device, like HTML. At first glance, this may seem clever, but imagine a scenario: the order system we discussed before includes a mobile app with a website. The website is for promoting the store, and the mobile app allows making orders and reading tutorials about the products. Some portion of managed data would have to be parsed to HTML, and the rest to some structure renderable within the mobile app. Using WebView would open that app to vulnerabilities, and if a different design is used for the web and app, the content in WebView would look out of place. As a result, traditional CMS solutions do not fit all usages. For another case, traditional CMS is a perfect choice, like an artist wanting to make his site on his own. Examples of popular traditional CMS solutions are WordPress and Joomla.

- **Headless CMS** - contrary to traditional CMS, this type does not offer managing UI but rather just CRUD operations on data. A streamlined feature set can enhance CMS efficiency within its intended scope of application. If the order system used a headless CMS instead of a traditional one, the system could have data managed for the website and mobile app from one point. This comes with some drawbacks, too; the UI cannot be defined in CMS, so the developers have to define it beforehand and then link their UI to back-end entities so everything will be rendered. Thus, the flexibility suffers; for example, when a new entity is added, the website/app has to be redeployed, and only the entity's data can be managed by the user. For the order system, this is a reasonable choice. Popular headless CMS solutions include Storyblok and Strapi.
- **Decoupled CMS** - a middle ground between traditional and headless CMS. It is similar to headless CMS but offers a front-end framework or templating language. The ability to manage UI makes it more flexible than headless CMS, and using some front-end framework makes it possible to implement more complicated UIs than traditional CMS. This compromise between two extremes can be a drawback; by providing the ability to manage UI, this type of CMS loses the ability to be used within the system with multiplatform clients.

In reality, CMS solutions often run in different modes; for example, WordPress can be tweaked to run as a headless CMS. This does not work and vice versa. Usually, headless CMS solutions cannot run as traditional ones.

2.1.2 Classification by purpose

- **Universal** - offers generic features that can be used for any purpose, but more developer work is needed for more advanced usage.
- **Specialized** - includes ad-hoc features for specific usage, optimal for one domain, useless for another. E.g. ecommerce CMS solutions.

2.1.3 Classification by platform

Before categorizing CMS solutions by platform, let us define what we mean by platform. The platform is the environment where the CMS is running. This includes the operating

system, processor architecture, programming language, database, object storage, and any specialized service (like LLM service/model). The platform can be cloud-based, like Google Cloud Run + Google Cloud SQL + Google Cloud Storage, or on-premise. In this definition of a platform, it matters in the cloud service, so stack using Google Cloud Run is a different platform than stack using Google App Engine, even though the rest of the services are the same. The difference seems negligible but impacts performance, scalability, and costs. Operating system and processor architecture are included in the platform definition because even though the CMS is written in the multiplatform language, some CMS solutions rely on OS-specific tools. The definition features a fuzzy part, „any specialized service,“ mainly for CMS solutions running in the cloud where specific hardware or capability is needed. The CMS server runs on rented CPU time and RAM, but for instance, running LLM is not well suited for CPU, but rather GPU or TPU. Cloud providers offer services for this kind of task. However, there are differences between individual services, and this can be another point where platform dependency can arise, whether it is that dependency on the AI model or the hardware it runs on, but this is beyond the scope of this thesis.

- **Managed** - the provider of the CMS hosts the CMS. It simplifies the deployment and maintenance but limits flexibility and scalability and can lead to vendor lock-in. This type of CMS is often proprietary and uses a SaaS model. Because they are written to run on a single platform, they can be optimized for that platform and thus offer better performance. Popular managed CMS solutions are Storyblok and Wix.
- **Single platform** - the CMS is hosted by the user on a specific platform. It offers more flexibility but requires more work to deploy and maintain. It does not eliminate vendor lock-in; e.g., if the CMS is developed specifically to work with AWS services, it cannot be effortlessly migrated to another platform. As well as managed CMS, this has the benefit of being optimized for the platform. An example of this type is FireCMS, which is designed to run on Firebase.
- **Multiplatform** - the CMS can run on multiple platforms, but some factors limit it to a limited set of platforms. That limiting factor is usually the database, but it can also be a programming language, object storage, or distribution method. Generally, when CMS relies on a database, it supports either SQL or NoSQL databases, but not both. CMS is represented by code in a particular language, but not all programming languages are supported by all cloud platforms; for example, AWS lambda does not support Golang. The multiplatform CMS solutions offer much flexibility but can lack optimization. Popular multiplatform CMS solutions are Strapi and WordPress.
- **Platform agnostic** - the CMS can run on any platform. This type of CMS is the most flexible and relies on powerful abstractions. The requirement of being platform agnostic means that the architecture of the CMS cannot suppose anything about the target platform, and also, the CMS has to be written in language that can be run on any platform.

2.1.4 Classification by availability

On the legal side, CMS solutions differ in the availability of their source code. In some cases this can be even deal breaker when choosing CMS, for example development website for Swiss government with a proprietary CMS would result in legal challenges¹.

- **Proprietary**
- **Source available**
- **Open source**

2.1.5 Classification by target audience

As mentioned before, CMS is a mechanism for user-friendly management of the content of information systems. Thus, the person using it is not required to know about the information system's tech stack. By target audience, this classification understands people integrating and deploying the CMS, not using it.

- **For users** - this type is easily configured/deployed, although it does not offer more advanced options for customization.
- **For developers** - configuration and deployment require knowledge about used technologies and thus can be more granularly configured. Some solutions even suppose that configuration includes reprogramming the logic of CMS.

2.2 Cloud based systems

To understand the benefit of being able to specify which cloud services to use for the CMS, we need to understand the architecture of cloud-based systems. First, let us define an information system: A set of interrelated components that collect, manipulate, and disseminate data and information and provide a feedback mechanism to meet an objective.[9]

Traditionally, servers operated as single, integrated applications, running directly on an operating system alongside necessary dependencies, such as databases for storage, which relied on the file system. This design, now referred to as monolithic architecture, has its characteristics: it is easily scaled vertically², but horizontal scaling³ is challenging[2]. Synchronizing state across multiple instances is complex, especially when an SQL database is involved, due to the difficulties of maintaining ACID compliance in a distributed environment.

In contrast, cloud architectures align with service-oriented architecture, although no single architecture can be generally applied to clouds. Instead it reconceptualizes the server as a collection of distinct services; computation, storage, and database functions are separated into individual services, each with its API. This design enhances scalability by allowing these services to scale independently. Although horizontal scaling of SQL databases remains challenging, the decoupling of services allows application servers to scale horizontally while

¹<https://datenrecht.ch/en/bundesgesetz-ueber-den-einsatz-elektronischer-mittel-zur-erfuellung-von-behoerdenaufgaben-embag-in-schlussabstimmung-angenommen/>

²by increasing the server's resources, such as adding RAM or upgrading the CPU

³adding more servers

maintaining a single instance of the database (a topic that falls outside the scope of this thesis).

This approach offers following advantages:

- **Scalability** - Services can be scaled independently, allowing for more granular control over resource allocation. For example, a service experiencing high traffic can be scaled horizontally by adding more instances, while other services with lower demand can remain unaffected.
- **Flexibility** - The decoupled nature of services enables agile development and deployment. Teams can work on individual services independently, accelerating development cycles and facilitating continuous delivery.
- **Resilience** - By isolating components, services can improve system reliability. If one service fails, it is less likely to impact the overall application.

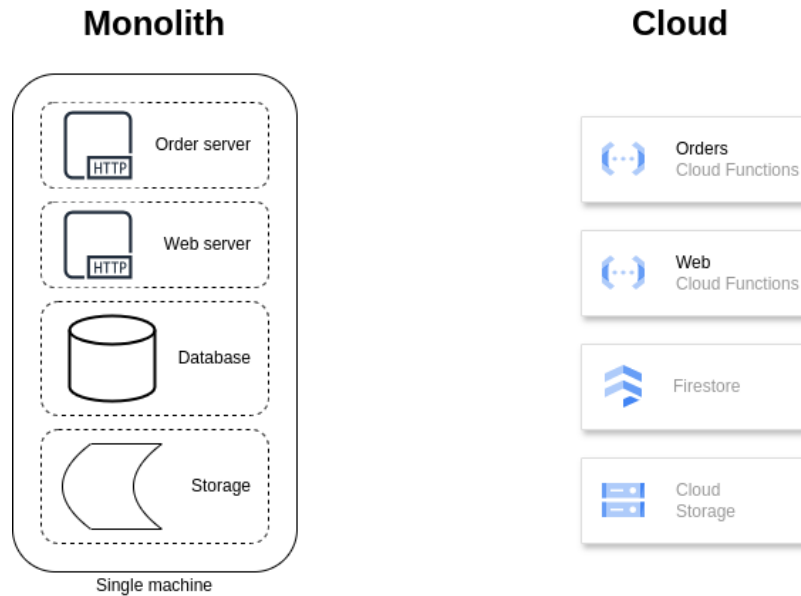


Figure 2.2: Architecture shift from monolithic to service-oriented (cloud)

2.2.1 Database and storage

Collecting data indicates that the data is stored persistently in the cloud, which means using a database or a storage service. There are multiple database and storage solutions, all of which have different characteristics.

Take, for example, these Google's database solutions:

- **Cloud SQL**⁴ - fully managed relational database service for MySQL, PostgreSQL, and SQL Server.
- **Cloud Spanner**⁵ - fully managed, scalable, relational database service for regional and global application data.

⁴Cloud SQL - <https://cloud.google.com/sql>

⁵Cloud Spanner - <https://cloud.google.com/spanner>

- **Firestore**⁶ - highly scalable NoSQL database for web, mobile, and server development from Firebase and Google Cloud.
- **Firebase Realtime Database**⁷ - NoSQL cloud database to store and sync data between users in realtime.

Each solution can contain user-managed data; the possibility of using CMS with that database solution is not ruled out. Under specific circumstances, some of those solutions are interchangeable, although their pricing is vastly different. Let us compare Firestore and Cloud SQL.

Firestore is fully managed serverless solution billed by⁸:

- The number of documents read, written, and deleted.
- The number of index entries read to satisfy a query.
- The amount of storage a database uses, including overhead for metadata and indexes.
- The amount of network bandwidth used.

And Cloud SQL is fully managed relational database service billed by⁹:

- vCPUs per time unit
- Memory per time unit
- Storage used per time unit
- Cloud SQL egress traffic and IPv4 address per time unit
- Instance type per time unit

Notice that Cloud SQL's pricing is billing for CPU time, unlike the Firestore; this means idle time will be billed as well. That means Cloud SQL for systems that expect much idling is cost-inefficient, and if the use case allows, Firestore would be a better choice.

2.2.2 Compute

Definition of information system mentions: manipulate and disseminate data. In other words, some logic has to be executed; in the case of cloud solutions, that means some computing service has to be used. Similarly to database and storage, there are multiple computing solutions, all of which have different characteristics. Like these Google's compute solutions:

- **Cloud Run**¹⁰ - fully managed auto-scaling platform for running stateless containers.
- **Cloud Functions**¹¹ - event-driven serverless functions that scale with usage.

⁶Firestore - <https://cloud.google.com/firestore>

⁷Firebase Realtime Database - <https://firebase.google.com/docs/database>

⁸Firestore pricing - <https://cloud.google.com/firestore/pricing>

⁹Cloud SQL pricing - <https://cloud.google.com/sql/pricing>

¹⁰Cloud Run - <https://cloud.google.com/run>

¹¹Cloud Functions - <https://cloud.google.com/functions>

- **App Engine**¹² - platform as a service (PaaS) for building and running applications on Google's infrastructure.
- **Cloud TPU**¹³ - custom machine learning accelerators for training and prediction.
- **IonQ Quantum Cloud**¹⁴ - quantum computing service.

As can be seen, some of the services are interchangeable, like Cloud Run, Cloud Functions, and App Engine; the rest are for specific types of computation. Programs like HTTP servers can run on any of the three services, yet the deployment process and runtime characteristics vary, and as with the database, the pricing is different.

2.3 Problem with CMS on cloud

Now, with an understanding of cloud-based systems and an overview of some cloud services, we can see the problem with existing CMS solutions.

Content Management System solutions existed long before modern cloud platforms. As a result, those solutions used monolithic architecture. For traditional CMS, monolithic architecture makes sense, as in most use cases of traditional CMS, the CMS is not being integrated into the existing system.

This does not apply to headless CMS solutions. Generally, when headless CMS is used, the reason is that either traditional CMS is not flexible enough or the system already exists. However, during development, it turned out that CMS would better manage some parts of the system. If CMS has to be integrated into the existing system, problems arise. The existing system has a back-end based on some technologies and is hosted elsewhere. Considering the number of languages, frameworks, databases, and platforms, it is evident that the probability that the system and CMS are using the same stack is low. This results in duplication, integration, and deployment challenges, and if the developed system is cloud-based, it may result in limited scalability and mixing design patterns.

Consider the previous example of the order system, which uses Firestore for all of its data, and it would be convenient to have blog posts stored in the same database. Sadly, CMS options that can be used with Firestore are limited. Strapi was chosen as the CMS, and the order system was forced to use Cloud SQL.

¹²App Engine - <https://cloud.google.com/appengine>

¹³Cloud TPU - <https://cloud.google.com/tpu>

¹⁴IonQ Quantum Cloud - <https://ionq.com/quantum-cloud>

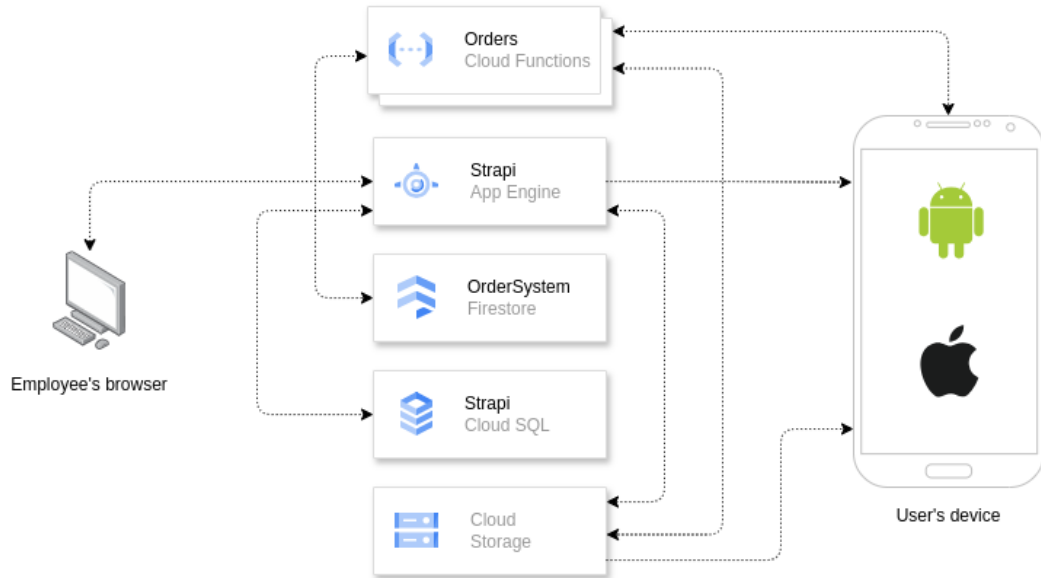


Figure 2.3: Example structure of Strapi alongside with order system

There are two back-ends, and Cloud SQL requires at least one instance running, billed by CPU and memory hours; this dramatically reduces cost-effectiveness. As noted in section 2.2.1, Firestore and Cloud SQL have different pricing models. By simple math, it can be inferred that using Cloud SQL, the cost is not less than the instance price per hour times 730 hours per month. For usage like blogs, the system needs a database to be operational anytime, thus, on average, 730 hours per month. Even with the slightest instance, db-f1-micro, and storage as low as 20GB, the cost is not less than 12€¹⁵. On the other hand, for a comparable price, we could have Firestore with 100GB of storage, 200 000 daily reads, 30 000 daily writes, and 1 000 daily deletes, and the cost would be less than 13€¹⁶. Cloud SQL would be more cost-effective only in case of high traffic, however, to justify the cost, the traffic would have to be for extended period, which is beyond expectation of the order system. At this price, it would make sense to consider writing ad-hoc CMS or embedding that functionality into the system, yet this results in more time spent on development and, therefore, higher costs for the client.

¹⁵Cloud SQL pricing calculator - <https://cloud.google.com/products/calculator?dl=CjhDaVE0TWpa a00ySm1NQzAyWW10akxUUXhaVGN0WwpjMFppMDFPVGt6WkdJMF1tWmhZemdrQVE9PRAHG1Q1MThDODM2QS1CMzA5LT QxNjMtOUE4Qy04MTc3MTEyMEZFMjk>

¹⁶Firestore pricing calculator - <https://cloud.google.com/products/calculator?dl=CjhDaVJqWmpBeU 1qTTFZeTFqWlRobExUUTRNVEFOWVRaak9DMWlabUpqWW1Sak5ETTJZemtRQVE9PRAXGiRDNjhCRENCNyOzQUMxLTQ1 QkMtQTEwNi0zMOJFNURFMzY4MUY>

Cost Estimate Summary

As of Nov 18, 2024 • 7:30 PM

Prices in EUR

DATABASES

€12.04

PostgreSQL (Cloud SQL)

€12.04

Service type

PostgreSQL

Instance-time

730 Hours

€8.52

Storage (Provisioned Amount)

20 GB

N/A

Instance-time

730 Hours

€3.52

Region

Frankfurt (europe-west3)

N/A

Cloud SQL Edition

Enterprise

N/A

Number of instances

1

N/A

Select SQL instance type

db-f1-micro, vCPUs: 1, RAM: 3.75 GB

N/A

Enable High Availability configuration

false

N/A

Storage Type

SSD

N/A

Committed use discount options

None

N/A

Total estimated cost

€12.04 / mo

Cost Estimate Summary

As of Nov 18, 2024 • 7:40 PM

Prices in EUR

DATABASES

€12.70

Firestore

€12.70

Service type

Firestore

Document Reads (per day)

200000

€1.65

Document Writes (per day)

30000

€0.33

Document Deletes (per day)

1000

€0.00

Total Stored Data (per day)

100 GB

€10.73

Location type

Region

N/A

Location

Frankfurt (europe-west3)

N/A

Total estimated cost

€12.70 / mo

The estimated fees provided by Google Cloud Pricing Calculator are for discussion purposes only and are not binding on either you or Google. Your actual fees may be higher or lower than the estimate.

Figure 2.4: Cloud SQL and Firestore budget comparison

The problem is not just the cost but also maintainability. Having to manage two back-ends means the need to know both; the developer cannot just unthinkingly follow the deployment tutorial without having underlying knowledge about used technologies. First of all, the developer has to understand the new tech stack and then gain experience, and in reality, that is while deploying the CMS and learning the hard way. This is not only time-consuming but also error-prone and can lead to security risks, which, in far-fetched scenarios, can lead to losing clients.

Also, the newly used services bring their own set of vulnerabilities and ways to exploit the system.

Generally, when using CMS on a cloud not optimized for the specific stack on which the system is running, the system may suffer from these problems:

- Duplication
- Difficult integration/deployment
- Scalability limitations
- Mixing design patterns
- Reduced ability to optimize costs
- Increased complexity
- Decreased performance
- More attack surface

Often, the solution to this is writing an ad-hoc CMS. This is a reasonable choice for companies constantly running on the same stack. Nevertheless, this is reinventing the wheel and not the solution to the original problem - missing the ability of existing CMS solutions to be ported to any platform (while keeping the efficiency).

Chapter 3

Requirements

After this brief overview, it is evident that there is no single „ideal“ CMS solution. Individual types within specific classifications of CMS are contradictory approaches to trade-offs that come with creating CMS solutions. Consequently, the ideal of CMS solution depends on the purpose of the CMS. For instance, a CMS made to make money would use a proprietary license and be managed instead of a self-hosted multiplatform solution. A great example of a CMS made for money is Storyblok. On the other hand, a solution driven by a community of volunteers would be open source and generally offer more deployment options, like Strapi. There are dozens of CMS solutions for different purposes, yet there is a gap in CMS solutions designed for the cloud that does not depend on specific technologies[7]. More specifically, no solution would allow running on any combination of cloud services on any cloud platform.

Before designing a new CMS, it is important to define the requirements for the CMS; we will use different classifications of CMS to define the requirements.

- **Architecture** - The CMS will be headless to ensure high flexibility.
- **Purpose** - As the industry gap is in CMS solutions for the cloud, there is a need to cover a big portion of use cases. Thus, the CMS will be universal.
- **Platform** - To ensure easy portability between individual cloud platforms, the CMS will be platform agnostic.
- **Availability** - Since the goal is to demonstrate that these kinds of CMS solutions are possible, the CMS will be open source so other CMS developers can inspire and move the industry forward.
- **Target audience** - This level of flexibility is needed mostly for more complex systems, which knowledgeable individuals develop. Therefore, the target audience is developers.

The primary problem that needs to be solved is the portability of CMS. The aim is to create a platform-agnostic CMS, which is nearly impossible, as programming language limits the platforms. However, creating a CMS that runs on any operating system and can run specific programming languages is possible. The term portability also means the ability to combine any database with any storage technology. To solve niche cases like needing storage on different cloud platforms than the rest of the system, the CMS has to support running on multiple cloud platforms simultaneously. Existing CMS solutions suppose that

developers integrating them will study the internal logic of CMS before integrating it. Sadly, developers are often under time pressure and do not have time to study multiple CMS solutions that best suit the system before choosing one. The new system must adapt to the existing system, not vice versa. This principle will also apply to deployment; the user will be required to have extensive knowledge about the platform, not the CMS. Every CMS has to be configured; most existing CMS solutions have some portion of configuration done on runtime, which is convenient. However, it prevents the configuration from being tracked using a version control system. The proposed solution must enable configuration entirely using code, and elements configurable on runtime must be serializable to code. There is a false premise that when some system uses a CMS, the developer must have an internet connection to develop that system while using CMS; this is not true, yet existing CMS solutions often require hosting before they can be used. The new solution must leave the option to use the CMS offline and then deploy it to the cloud. Regarding content management capabilities, the CMS will have the ability to manage data in the database and file storage, and for more specific usage, it will provide component API. Often, users manage the content on mobile devices, so the CMS must be easily usable on mobile devices.

3.1 Managing database content

Some CMS solutions (e.g., Storyblok) focus exclusively on providing a way of creating components and merging them into pages. For a system that needs only one set of data to be manageable, that kind of functionality would be overkill and would introduce unnecessary complexity. Having those data stored in a database table/collection would be much simpler. Additionally, the ability to manage database content would create a new use case for CMS, which would involve managing the internal data of an information system. Therefore, managing content directly in the database is a vital feature of the CMS. Managing content includes manipulating primitive data types and complex ones commonly used to write an article (it does not include creating/deleting database collections/tables). Those types include:

- Boolean
- Number
- String (limited length)
- Text (long string)
- Date
- Markdown
- Storage reference (also multiple)

These types can be further specified, such as the maximum length of string, step size of number, and others. Types of individual properties, as well as their constraints, must be configurable. As mentioned before, the existing CMS solutions require their database for the content, which implies that the database is the only specific type of database. The new solution must be able to communicate with any database and manage multiple databases at once. This feature will be handy for systems undergoing migration from one database

to another, where the new system parts will be stored in the new database, but until the migration is complete, the old database will be used for the old features. This feature is necessary for migration, as the existing CMS solutions use their own database, which is untouched during system migrations since there are two back-ends. The migration is happening only on the system's back-end. So, being able to communicate with multiple databases is not a new ability but rather a measure to keep feature parity with existing CMS solutions. Still, this measure introduces new use cases.

The contribution of this solution for managing database is:

- Ability to manage collections from multiple databases
- Adaptability to different database solutions
- Option to communicate with databases on different platforms at once

3.2 Managing file storage

Static assets, like images, videos, and others, are undividable parts of the system's content. The CMS must provide a way to manage these assets, not on just one bucket, as usual, CMS solutions do, but on multiple. Working with multiple buckets gives developers more flexibility in dividing them into different classes. Imagine a scenario where the information system has two information feeds: one is for the general public, and the other one is for members of the owner company; when hosting assets for internal feed, the company needs to keep the assets inside of their company, not to leak them to the public. For this feature, separation of buckets is required; the public feed needs assets accessible without auth, whereas the internal feed hidden behind auth can source from the private buckets.

Managing file storage means:

- List buckets
- Upload files
- View files/directories
- Delete files
- Move files
- Create directories
- Delete directories

To avoid confusion, the manager will be familiar with file managers on operating systems. The storage manager can work with multiple buckets hosted on different platforms.

The contribution of this solution for managing storage is:

- Ability to manage multiple buckets
- Adaptability to different storage solutions
- Option to communicate with buckets on different platforms at once

3.3 Component API

For a use-case where one needs to manage bulk repetitive data, storing it in a database collection/table is enough; on the other hand, unique content that is displayed on just one page and the content is more complex, like UI components, the use of component API is more suitable. First, let us define what component API is, as some solutions are lacking it, and this CMS feature is not well known. The Component API enables the manipulation of components and the pages they form within the CMS. It corresponds to predefined UI components in the application that interact with the CMS. The standard procedure for using it is to define the component in the codebase; the component can have attributes and slots (sub-components); next, the defined component needs to be registered within the CMS (creating component using the API); after that, the component is ready to be used on a page. Creating a page is the opposite of creating a component; the developer creates a page in the CMS (at this point, the existing components can be used), and then the developer defines a route within the app that will request the data from the component API and render the page. By creating the front-end components in the code and loading page structure and their data from CMS, the component API doesn't create dependency on a specific front-end framework, which is the goal of headless CMS solutions.

The required features of component API are:

- CRUD components - managing components and their attributes; every component has a name and attributes. Attributes can have various types, the same as the types for managing database content, and one special type, which is a slot for sub-components. Additionally, changes in component structure must be tracked so that the changes can be reverted.
- CRUD pages - managing the hierarchical structure of components and attributes of a page. The page has its name and base structure specified by component. If the page is a website accessible to the Internet, the CMS will have WYSIWYG (What You See Is What You Get) capability. Same as with components, changes in page structure must be tracked.

Since one of the roles of this CMS is to be proof of concept (mostly its flexibility), there is an opportunity to do experiments. Previously, we mentioned decoupled CMS, which features some front-end framework. However, that feature makes the system dependent on a particular front-end framework. In the future, a feature could be implemented that allows custom components to be written in the CMS while keeping the system independent of the front-end framework. For now, this is not a hard requirement but a suggestion for experiments.

3.4 Summary of requirements

The new solution mostly focuses on non-functional aspects of CMS, while the functional requirements are mostly to keep feature parity with existing CMS solutions. The CMS market is many solutions that race to provide more features but overlook the non-functional requirements; that is why one of the aims of this solution is to be proof of concept that CMS with these requirements is possible. These requirements do not mention any modules or plugins because, due to other requirements, the CMS will be complex; adding more complexity could prolong the development time and potentially lead to an unsuccessful finish.

Non-functional requirements for the CMS:

- Platform agnostic
- Easily scalable
- Code-first configuration
- Offline capability
- Mobile friendly

Functional requirements for the CMS:

- **Managing database content** - managing collections
- **Managing file storage** - managing buckets
- **Component API** - components, pages, and optionally coding components

Chapter 4

Proposed solution

When we take the number of new requirements, it is clear that creating a new CMS from scratch is more manageable than integrating the requirements into the existing CMS solution. Since the new CMS will be open-source, it is important to make it appealing to the public, and two factors that affect the appeal are license and name. The choice of license is important, as it might be a deal breaker for some users. First, the author does not want to be liable for any damage caused by the CMS. This CMS is meant to be used by developers, and because some of them need to extend the functionality of the CMS, copying and modifying are allowed. Additionally, one may need to create and maintain a fork of the CMS. Thus, the right to distribute the CMS is important. Considering we are developing a CMS, the system will be part of a greater, often proprietary system.

Licenses that meet these requirements are MIT, Apache 2.0, MPL 2.0, and BSD 3-Clause. Considering this CMS's philosophy to be as flexible as possible, the most permissive license was chosen, the MIT license.

A less significant factor is the name; it was chosen to be an analogy to the goals of the CMS. Genoa¹ is a type of sail located at the front of the ship; the analogy is that the whole ship represents an organization, the hull translates to tangible and intangible assets of the organization, and the rigging translates to software moving the organization forward, where mainsail represents their central information system, mast their tech stack, and genoa is the CMS. Usually, when sailing, the mainsail is hoisted first, and often when developing an information system, the system is developed first, and then the CMS is integrated, same as hoisting genoa after the mainsail is hoisted. This analogy is the reason behind the name GenoaCMS. The code will be stored on Github, with a multi-repository structure, rather than a mono repo. This is because some libraries that need to be created for the CMS can be used for projects unrelated to the CMS, and keeping the history separated makes sense.

4.1 Ensuring portability

According to the requirements, GenoaCMS must be platform agnostic, thus supporting any database, storage, and computing solution. Earlier, it was mentioned that platform agnostic solutions are instead a theoretical utopia rather than reality; programming language - Javascript^{4.3.1}, will be the limiting factor. Therefore, GenoaCMS will have limited usable computing platforms for only those supporting Javascript runtimes. Where the new solution is not constrained is the database and storage solutions. One CMS cannot be universal, but

¹Genoa - [https://en.wikipedia.org/wiki/Genoa_\(sail\)](https://en.wikipedia.org/wiki/Genoa_(sail))

thanks to bridge pattern, we can decouple the logic of the CMS from the implementation details of the database and the storage solutions[3]. The mechanism for translating the CMS operations to platform-specific operations will be called an adapter. The CMS operations will be specified in the individual „cloudAbstraction“ package described by Typescript declarations.

4.1.1 @genoacms/cloudabstraction

Typescript package declaring all abstract functions for the CMS. It is an abstraction layer above the cloud services, implemented by adapters used by the CMS. This package describes the format of the configuration file and individual services. Initially, the services were only database, deployment, and storage; later, authentication and authorization were added.

Making database abstraction is problematic, as there are many types of databases, all with different characteristics. Some of the types differ in level, where they cannot implement all the operations needed for CMS. The operations used by CMS need to be reduced to a level where relational and NoSQL databases can implement the operations. The goal of GenoaCMS is not to support communication with every type of database. For example, pairing a CMS with a vector database would be impractical and outside the intended scope of the system. Because of this, a subjective decision needs to be made to define which operations are needed for CMS. For example, requiring adapters to implement complex queries with aggregations would make managing databases simpler. However, it would result in an inability to use some databases (like NoSQL). A potential solution would be limiting features on less capable databases. Nonetheless, this would compromise platform agnosticism. In such a case, the GenoaCMS would have different feature sets on two different adapters, which defeats the purpose of the CMS - having the same experience on all platforms. Because of these constraints, the database operations are limited to:

- Create a document
- Get collection documents (or its subset)
- Get a document
- Update a document
- Delete a document

For the database service, we will use document-oriented nomenclature. The document is a heterogeneous data type of key-value pairs. The collection is a homogeneous data type of n-documents. Thanks to the simplicity of the supported operations, they can be implemented by a broad range of databases, including SQL, document, object-oriented, and more. Using a configuration file with handler method definitions of those operations is possible. By similar shoehorning, GenoaCMS can be used even with types of databases not designed for such use, like key-value stores or cache solutions in far-fetched scenarios. As some databases (e.g., DynamoDB) require information about the shape of the data, the GenoaCMS requires specification of the document schema for collection. By design, collections are registered in the configuration file; the collection has its name, schema, and primary key (name of the key and its schema).

Similarly to a database, the types of storage solutions differ, and the feature set has to be reduced. In this case, the differences are less in the data structure than in data access.

Usually, all storages share a tree-like structure. The adapter generally uses its API when working with cloud storage services; in contrast, it uses file system API when working with local file systems. Thus, the operations must be reduced to the intersection of both APIs. The operations are:

- Get an object
- Get a signed URL
- Get a public URL
- Upload an object
- Delete an object
- List directory
- Create a directory

Since the storage service is essentially a filesystem with the differences being structured to the collection of buckets, thus all of these operations are localized to the bucket. The buckets can have different access policies, and adapters must respect them. Using buckets is native to cloud storage services but not for the filesystem; adapters for the filesystem will have to emulate the buckets by using directories with optionally varying permissions. Cloud services often provide API methods to get object URLs. Getting URLs in other environments is more complicated, as the adapter must have information about the storage address; obtaining the information is implementation-specific. Adapting this cloud-focused structure (buckets) to the filesystem is not a problem, unlike adapting the filesystem to a structure usable on the cloud; this is why the bucket-focused structure was chosen.

Managing access to the CMS is a crucial security feature. While considering the security and requirement of using a shared back-end, the GenoaCMS must take a decoupled approach to authentication and authorization. The problem is that some systems have their authentication handled by an external service, database, or combination of different services. For example, Firebase Authentication offers an end-to-end authentication solution but lacks authorization that Firestore or even IAM could handle. When evaluating the number of authorization and authentication service combinations, the CMS cannot group them, as replacing one of the pairs would mean writing a new adapter. As a result, the CMS has separate adapters for authentication and authorization.

An authentication adapter is required to implement the following:

- Check user-password combination

The authorization adapter is responsible for the following:

- Check user's permissions

Although the adapters are separate, the CMS supposes that the username for a single user is the same across all services; that is because the adapters are decoupled only for flexibility reasons and intended to act logically as one service.

Package „@genoacms/cloudabstraction“ also provides type declarations and unit tests for the adapters. Having one set of tests for all adapters ensures CMS consistency on different platforms. Having these unit tests makes even more sense when considering these facts:

- The adapters require their configuration (adapter specific); when writing an adapter suite (package providing all adapters necessary for one platform), the developer writing the adapter can write a config for a system where the CMS with the new adapter will be used, and the very same config can be used in unit tests. This means that the unit tests are not only testing the adapter but also the configuration. This also has a negative side; for adapters with poor configuration documentation, the adapter and its configuration mismatch can seem like a bug in the adapter. This means adapters should embrace proper error handling.
- Adapter packages having their testing configuration public can be used as examples for developers integrating the CMS.
- Unit tests set grounds for performance benchmarking.
- Unit test serve as additional documentation for the requirements of the adapter.

4.1.2 Adapters

Adapters implement service methods based on package „@genoacms/cloudabstraction“ requirements. The adapters are responsible for translating the CMS operations to platform-specific operations. Due to the number of existing platforms and their combinations, an author cannot implement and maintain adapters for all of the platforms. It is expected that the community will write the adapters. Consider entity A; it can be an individual or an organization developing systems on its own tech stack; by focusing on a specific tech stack, the entity probably has the capabilities to write a code that maps GenoaCMS’s operations to its platform of choice. The biggest challenge for entity A is understanding the requirements of the CMS. In case of lacking skills of entity A, for popular platforms, there will probably be entity X that will succeed in writing the adapter. For niche platforms, where there is no entity B, entity A has to face the challenge or give up having the CMS on the same tech stack as the rest of the system.

An adapter is a code that implements all operations for a given service from the list of services in package „@genoacms/cloudabstraction“ and passes all of its tests. When distributing an adapter (e.g., on npm), it is recommended that its version be matched with the version of „@genoacms/cloudabstraction“ it is implementing.

Adapter suite

Most platforms provide multiple services, such as Google Cloud Platform, which provides databases, storage, computing, and more solutions. There is an assumption that when the developer uses some service from one platform, it uses more services. „Adapter suite“ represents a package grouping multiple adapters for one platform.

One caveat when creating/using an adapter suite is that the adapter suite can provide at most one adapter per service; this means adapter suites are created with specific services in mind rather than offering multiple options for database, storage, and rest.

Initially, when GenoaCMS development started, two adapter suites were created for GCP (Cloud storage, Firestore, Cloud Run, IAM) and AWS (S3, DynamoDB, Lambda, IAM).

4.2 Scalability

One of the biggest focuses of the cloud is scalability. Since the CMS is assumed to be run on the cloud, scalability is inherently a requirement. Nevertheless, in some cases, the CMS may not be hosted on the cloud, and this can significantly reduce scalability, like running it from a single Raspberry Pi. As it is apparent, the scalability is directly influenced by the target platform; this leads back to adapters, as one of their responsibility is not to limit scalability.

Before settling on how scalability will be achieved, it is important to distinguish between the meanings of scalability and its value to GenoaCMS:

- **Compute scalability** - determines how many requests can be handled by the CMS at once. This factor is important for a lot of existing solutions that provide APIs to access their content. Using content APIs seems intuitive, but it has a few significant drawbacks. First is the requirement to have an additional service running to process the requests; for clouds, that means stacking APIs on top of each other (as cloud services have their own APIs). The second drawback is the need to document the API for users. The third drawback is the performance bottleneck that can occur while running the API due to the hardware it is running on. GenoaCMS has a different approach; the individual services can run on different devices. Thus, every service has its API and is scaled independently. This approach is cloud native. By not using content API, which processes the data on-demand, GenoaCMS has to process the data before it is requested. For data storage and databases, this is not a problem, as cloud services provide their APIs; instead, this makes implementing component API a challenge. Because the need to not involve GenoaCMS computing in handling component API requests, the storage service implements the component API. When creating a component or a page, the data are saved to storage, and the storage service provides the API to access the data. It is important to have the data saved in an easily parsable format [4.7.2](#). This approach is scalable and allows for easy caching, and the storage service handles the scaling. This approach results in GenoaCMS computing being active only when users manipulate the data, not when reading it. Considering that not many users are expected to manipulate the data simultaneously, no special attention is needed to scale the computing; even the smallest instance offered by the cloud providers should be enough. This heuristic lowers costs, as the CMS is not billed for idling.
- **Data volume scalability** - this aspect is related chiefly to non-cloud-based systems; cloud storage solutions do not impose any limits on the data volume. Because of this, the CMS is completely ignoring the problem of checking bucket used/free space. Thus, adapters to non-cloud-based systems must handle this problem independently. The following important requirement for adapters is to allow uploading large files, which can fail for reasons like low timeout when communicating with the storage service. Databases are handled the same way; GenoaCMS supposes that the database has enough resources to handle the expected data volume, and adapters do not introduce any limits. Cloud platforms usually ensure these properties; for non-cloud-based systems, users are expected to handle it on their own.

- **Data access scalability** - latest trends indicate that servers are being placed closer to their users, it is called edge computing². In other words, edge systems operate in multiple locations, even worldwide (like CDN). For GenoaCMS, this results in a requirement to communicate with multiple databases and storage servers at once. This is not usually a problem for the cloud, as cloud service APIs allow working with a database or storage distributed across multiple locations. For non-cloud-based systems, where a user of the CMS handles its infrastructure, this means either implementing similar APIs to cloud services (in fact, this approach is more practical; more about this in the use-case section) or connecting the CMS to multiple databases/storage servers. Because of this, GenoaCMS uses providers API^{4.5.1} to communicate with multiple databases/storage servers simultaneously.
- **Diversity of platforms** - during the expansion of the information system, it may happen to be needed to expand to a different cloud provider, for instance, a fictional system serving as a digital exhibition, originally hosted on Google Cloud Platform, which lacks data centers in China, Russia and has very few of them in Africa and South America³. Sadly, there will probably never be a cloud platform uniting the whole world; this means that globalized systems will need to run on multiple platforms simultaneously, like a combination of Google, Yandex, Alibaba, and other clouds. The GenoaCMS must allow the use of services from multiple platforms and even the same service from multiple platforms at once. Because of the assumption that database and storage content are the same among multiple platforms, there is an inherent need for synchronization. Like data access scalability, synchronization is handled by the „providers API.“ During the configuration of CMS, the user should group databases/buckets to mark them as linked, so when operations are executed on some data, the changes are distributed all across linked providers. However synchronizing data between platforms comes with great challenges, like error handling, retrospective synchronization of data edited outside of the CMS. Because of this complexity, data synchronization is not part of initial GenoaCMS release.
- **Diversity of services** - by allowing using services from multiple platforms, the CMS must be able to handle different services because there is no feature parity between all services on all platforms. This factor gives users flexibility in using which service they want and is also a powerful scaling tool. Consider the fictional product tracing system, which was initially used for a tiny country in the EU. Eventually, the system is expanded to the whole EU; in the first stage, one powerful Cloud SQL instance could satisfy the system's needs; later, as it expanded, the system required a distributed database, like Spanner. When maintaining a system of this size alongside a content management system, the CMS should not be a limiting factor; the ability to switch between services is crucial. Thanks to the bridge pattern, migrating CMS from one service to another is a matter of swapping the adapter, although users have to move the data by themselves. GenoaCMS assumes their users have other data on their services that are not affiliated with CMS. When making significant changes like migration, the users should take all responsibility for the migration rather than relying on some tool of CMS.

²edge computing - https://en.wikipedia.org/wiki/Edge_computing

³GCP locations - <https://cloud.google.com/about/locations/>

4.3 Selecting tools

The proposed solution is complex and includes many abstractions, as evidenced by requirements. The architecture of CMS covers both the server and client sides, so when selecting tools, it is important to consider both. There is an important choice of server and client-side programming language and optional frameworks and libraries. For a back-end language, there are many options; on the other hand, the languages supported by browsers are limited only to JavaScript and languages compilable to WebAssembly⁴ (e.g., C, C++) or JavaScript (TypeScript, Dart, Elm).

A robust cloud-focused CMS requires a language and framework combination that provides the following:

- **Cloud-Native compatibility** - Cloud platforms offer various compute services, of which programming language support is limited (mainly to a few major languages used for web development). Since GenoaCMS aims to be as flexible as possible, the availability of cloud platform SDKs and computed services is a primary factor in selecting the programming language.
- **Type safety** - Given that the behavior and data of GenoaCMS are defined by configuration, the character of the CMS is generic; for that to be possible, the system includes a lot of abstractions and auxiliary structures. A type system is a crucial mechanism for enforcing design decisions at compile time, preventing many potential errors before they can manifest in the running program[6].
- **Rich ecosystem** - Building a CMS from scratch includes solving a vast amount of more minor problems, like building UI, data validation, data management tools (e.g., Markdown editor), and others. In order to develop the system in a reasonable time, the ability to use existing solutions is important.
- **Strong user base** - As mentioned before, GenoaCMS expects the community to write adapters, and the community is more likely to write adapters for languages they are familiar with; thus, using a popular language is a benefit.

4.3.1 Programming language

The language selected for the development of GenoaCMS was Typescript (Javascript).

The primary criteria for choosing a programming language for GenoaCMS are platform independence and strong cloud support. The most common runtimes on cloud platforms are Node.js, Python, Go, Java, Ruby, PHP, and .NET. Nonetheless, more options were considered, but for consistency, only notable ones will be mentioned in this comparison. In this comparison, the factors are based on the requirements of the CMS; factors like syntax or author familiarity are omitted.

⁴WebAssembly - <https://webassembly.org/>

Language	Cloud support	Ecosystem	Type checking	Ease of abstraction	Performance
C	Limited	Mediocre	Mediocre	Limited	Excellent
Python	High	High	Limited	Excellent	Limited
Java	Mediocre	High	High	High	High
PHP	High	High	Limited	Mediocre	Mediocre
JS (TS)	Excellent	Excellent	Excellent	High	High

Figure 4.1: Simplified comparison of programming languages

The factors that influence the selection of a programming language are inherited from the factors mentioned above. When rating the factors for individual languages, the following terms were used: Excellent, High, Mediocre, and Limited. These terms reflect more detailed information about the language written in individual sections.

Term cloud support refers to the availability of SDKs for cloud services and the number of services supported. It also covers the ability to run on cloud platforms. The perfect language would have available SDKs for all services on all platforms and would be executable on any compute service. This factor was evaluated by checking the availability of SDKs for the most popular cloud platforms (AWS, GCP, Azure) and the number of services supported by the SDKs.

Ecosystem refers to the number of libraries and frameworks available for the language that are relevant to the problematics of information systems and web development. Evaluation of this factor was done by comparing a number of claimed libraries on package managers (e.g., PyPi, Maven).

Type checking refers to the language’s ability to check types at compile time. The perfect typechecker would allow for defining custom types and complex nested data structures, including generics, and having a good type inference system. This factor was evaluated by previous experience with the language and by checking online resources about typecheckers.

Ease of abstraction refers to the language’s ability to create abstractions. The perfect language would allow for creating complex abstractions with minimal boilerplate code and responsibilities. This factor was evaluated subjectively by trying to write complex structures within the language and comparing the code.

Performance was not compared specifically for web server workloads, as public benchmark workloads often do not reflect GenoaCMS. Instead, the reference benchmark[5] was based on N-Queens problem.

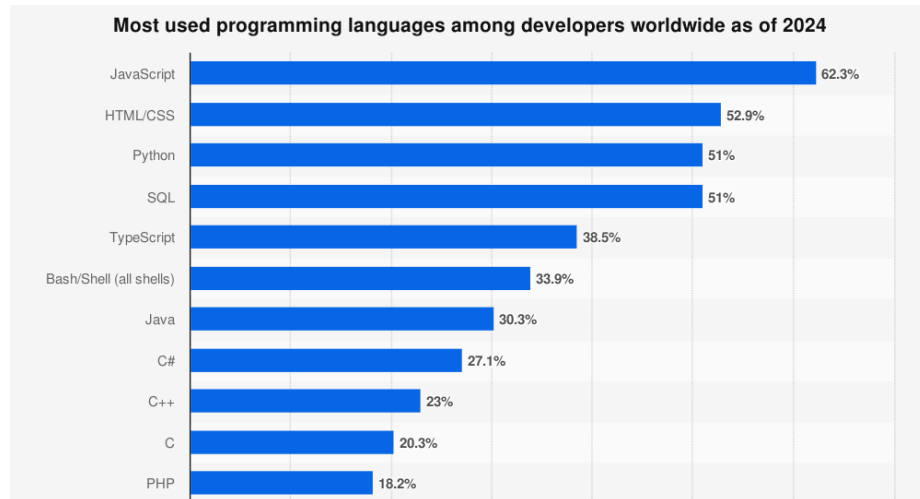


Figure 4.2: Most used programming languages in 2024 according to Statista <https://www.statista.com/statistics/793628/worldwide-developer-survey-most-used-languages/>

Factor popularity was not included in the comparison, as it is not a factor that affects the CMS directly. However, it still has value; it can indicate the size of the set of users that can write adapters for the CMS.

C

C is not a common choice for a web application's back-end (nor front-end), like a CMS. Nevertheless, it is worth considering, as it is one of the most popular languages and offers other advantages.

Pros:

- **Highly portable** - another benefit of compiled language is that the code can be compiled to run on any platform, which is important for GenoaCMS, as it aims to be platform agnostic.
- **Solid ecosystem** - C is one of the oldest programming languages with a vast ecosystem. Nevertheless, web development is not the primary focus of the C programming language. That results in a small amount of web frameworks.
- **Possibility to share code with front-end** - in the case of using C on the back-end, there is an option to use it on the front-end by compiling it to WebAssembly.
- **Small overhead** - thanks to being a compiled language and not having runtime mechanisms that consume compute time (like garbage collector), C is very efficient, which can result in lower billed time. Nevertheless, the cost-efficiency of GenoaCMS is ensured by architecture and adapters, not by language.

Cons:

- **Low-level nature and limited abstractions** - Requires manual memory management, lacks built-in data structures, and problematic use of unions without discriminator makes using C for web development potentially error-prone and time-consuming.

- **Unsatisfiable type system** - mainly because of lack of generics, the behavior of pointer types, and how the system handles type unions[8] which are often used for systems such as CMS.
- **Reinventing the wheel** - Many of the core functionalities of a CMS (e.g., form parsing and validation) have already been well-solved in other languages and frameworks.
- **Limited potential for writing adapters** - As can be seen in the figure 4.2, C has a vast user base, but compared to languages like Javascript, Python, or Java, it is in a disadvantage.

Writing the CMS in C (or similar compiled language) and then compiling it to run on any platform would be possible. However, this would create challenges, namely dealing with the low-level nature of C and the need to reinvent the mechanism existing in other languages/frameworks. The following counterargument for C is the need for many abstractions for making which C is not the best choice. A content management system is not an application requiring high performance, and C usage could be considered premature optimization. For these reasons, using C is ruled out. Some of the problems would be solved by using C++. However, the significant disadvantages are shared by both of the languages.

Python

Python has become a popular choice for web development. It was considered for GenoaCMS because of its popularity and the number of cloud platforms that support it.

Pros:

- **Widely supported by cloud platforms** - Most cloud platforms offer Python runtime and have SDKs for their services.
- **Easy to abstract**
- **Rich ecosystem** - an extensive number of libraries for web development, data analysis, AI, and scripting⁵. However, it is still inferior to JavaScript's ecosystem for the web.
- **Option for type checking**
- **Highly popular among developers** - According to Statista, Python is the second most used programming language in 2024^{4.2}.

Cons:

- **Performance** - In the reference benchmark, Python was the slowest, taking 32.53 seconds to solve the N-Queens problem, while JavaScript (Node.js) took 0.74 seconds.

Java

Known for its robustness and cross-platform compatibility, has a long history in enterprise-level applications. However, it is not as popular as Python or JavaScript in web development thus its cloud support is lower. It was considered for GenoaCMS because of its strong type system and the number of cloud platforms that support it.

Pros:

⁵PyPi - <https://pypi.org/>

- **Easy to abstract** - Automatic memory management, includes built-in data structures.
- **Built-in type checking** - Statically typed, allows for complex nested data structures, includes generics.
- **Solid ecosystem** - Offers a wide range of libraries and frameworks⁶, although the number of those focused on web development is lower than for Python or JavaScript.

Cons:

- **Enforced object-oriented programming**

PHP

The historically dominant choice for web and CMS development. It was considered for GenoaCMS because of its web-focused ecosystem.

Pros:

- **Web-Specific Ecosystem** - tailored for web development with popular frameworks like Laravel.
- **Well supported by cloud platforms** - a solid number of cloud platforms support PHP and have SDKs for their services, although some platforms do not offer SDKs in PHP.

Cons:

- **Performance** - In the reference benchmark, PHP was the second slowest; it took 18.16 seconds to solve the N-Queens problem, although when running a just-in-time compiled version, the calculation took 3.53 seconds.
- **Type system** - despite improvements (e.g., type declarations), PHP still lacks the robust type systems seen in Java or TypeScript

⁶Maven - <https://mvnrepository.com/>

Javascript

For front-end development, Javascript is the industry standard, but its usage on the back-end is not as traditional as PHP. It was considered for GenoaCMS because of its rich ecosystem, strong type system, and the number of cloud platforms that support it.

Pros:

- **Widely supported by cloud platforms** - Similar to Python, Javascript is supported by most cloud platforms and has SDKs for its services.
- **Rich ecosystem** - has the biggest single language code repository⁷, which also includes the majority of web frameworks
- **Option for type checking** - TypeScript
- **Possibility to share code with front-end** - While using Javascript on the front-end (industry standard), the portion of code can be shared between the front-end and back-end. In the case of using TypeScript, this can enhance type safety, as there is no need to declare types twice.
- **High potential for writing adapters** - according to Statista, JavaScript is the most used programming language in 2024^{4.2}. Thus, the likelihood of finding a developer to write an adapter is higher than for other languages.

Cons:

- **Odd behavior** - Built-in feature like operators have inconsistent behavior⁸, which can lead to bugs.
- **Fragmentation** - Javascript has two module systems (CommonJS and ES6 modules), which can cause problems when integrating libraries.

While all other considered languages offer a wide selection of libraries, most web development libraries are for JavaScript and PHP. When selecting a language, decisive factors were the number of cloud computing services in which the language is supported, the number of SDKs for cloud services, and the back-end/meta-framework.

4.3.2 Framework

Writing a server and client for CMS includes various problems, like parsing requests, routing, handling sessions, rendering UI, and others. Those problems are not specific to CMS but to web apps generally, and they are well-solved by existing frameworks. As with the selection of language, there is a significant factor involved, and with the ambitions of GenoaCMS and time concerns, avoiding using the framework would seriously delay the MVP. Because of two-layer architecture^{4.4.1}, we want to separate cloud-specific logic with elevated privileges from the rest of the system; this factor creates requirements for rendering techniques. To achieve this separation, the client and server code has to be divided; in the case of using CSR (client-side rendering), we must create an additional API where the client can

⁷NPM - <https://npmjs.com/>

⁸<https://github.com/denysdovhan/wtfjs>

interact with the platform-specific endpoints. In contrast, frameworks using SSR (server-side rendering) offer the back-end API out of the box with 1:1 mapping to the front-end. The next factor supporting the use of SSR is security; shipping the code of adapters to the user's browser is a security risk; in the case of using CSR with API for CMS's operations, a new layer of authentication and authorization is required. To avoid using another API and thus a layer of security, the intuitive choice would be using code splitting and sending the code for CMS operations only to authenticated users, but the problem is that the authentication requires the code for CMS operations. This leaves us only with two choices, either SSR or CSR with additional API. The usage of SSR was chosen as it simplifies the architecture, and the availability of frameworks with bootstrapped SSR is high. After settling on language with a strong type system, in order to keep the benefits of the type system, the used framework must be type-safe. When selecting a framework, both back-end and front-end solutions must be considered. There is a plethora of JavaScript frameworks for both back-end and front-end, and we cannot consider all of them and their combination. Because of good integration and streamlined development process, it was decided to use a meta-framework solution that combines back-end and front-end frameworks. The most notable meta-frameworks are Next.js⁹ (React), Nuxt.js¹⁰ (Vue), SvelteKit¹¹ (Svelte). All of these share many similarities, and their recent versions made their differences even smaller (compiler for React¹², runes for Svelte¹³). The back-end parts of these frameworks also share many similarities, like filesystem routing, server-side rendering, and utilization of code splitting. Since these frameworks offer SSR and are type-safe, the choice was made based on the properties of the front-end framework. SvelteKit was chosen, as it uses Svelte on the front-end, which has the best style management, superior performance and smallest bundle size in the majority of cases^[4] (bundle size scale differently, as Svelte is not a framework which loads to client, rather a DSL compiled to JS).

4.3.3 User interface

To build the UI, a choice was made to use the CSS framework and component library based on it to speed up the development. The CSS framework was chosen Tailwind CSS, mainly because of its feature set and granular control over styling (compared to frameworks like Bootstrap). For Svelte, there are a few notable component libraries based on Tailwind CSS, such as Skeleton and Flowbite. The choice was made to use Flowbite, as it offers a comprehensive set of components, some of which Skeleton lacks or provides them with an underwhelming state (namely, modal component).

4.3.4 Data scheming and validation

The next significant facility of GenoaCMS is the data scheming library and validator. Since the whole stack uses JavaScript and the internal data of CMS (not the data it manages) are JavaScript objects, the most suitable option is to use a JSON schema. JSON schema is a JSON-based format for describing the structure of JSON data^[11]. It was preferred over JSON type definitions¹⁴ because of its extensibility, which is used for defining the

⁹Next.js - <https://nextjs.org/>

¹⁰Nuxt.js - <https://nuxt.com/>

¹¹SvelteKit - <https://svelte.dev/>

¹²React compiler - <https://react.dev/blog/2024/10/21/react-compiler-beta-release>

¹³Svelte runes - <https://svelte.dev/blog/runes>

¹⁴JTD RFC 8927 - <https://datatracker.ietf.org/doc/rfc8927/>

managed data by CMS in configuration files. For validation of the data was chosen library Schemasafe¹⁵, initially was decided to use Ajv¹⁶, later it showed up, that it is lacking ESM version was failing builds, Schemasafe was drop-in replacement for Ajv as it uses the same schema standard. Library Superforms¹⁷ was chosen to bootstrap the process of form communication between client and server. By default, Superforms uses Zod for validation but offers the option to use other validation libraries, and both Schemasafe and Ajv are supported. However, the integration of Schemasafe is more straightforward, which makes another fact supporting the decision for the transition from Ajv to Schemasafe. To clarify data validation of GenoaCMS, the system validates these events:

- Configuration file parsing
- Form submission (e.g., manipulating a document, page, component)
- Reading saved internal data (e.g., component definition)

4.4 General architecture

As already stated, the goal of GenoaCMS is to provide as much flexibility as possible and to be platform agnostic while having a consistent feature set. GenoaCMS assumes it is being used alongside other applications on top of existing infrastructure. Consider the following figure representing the general architecture of GenoaCMS in production.

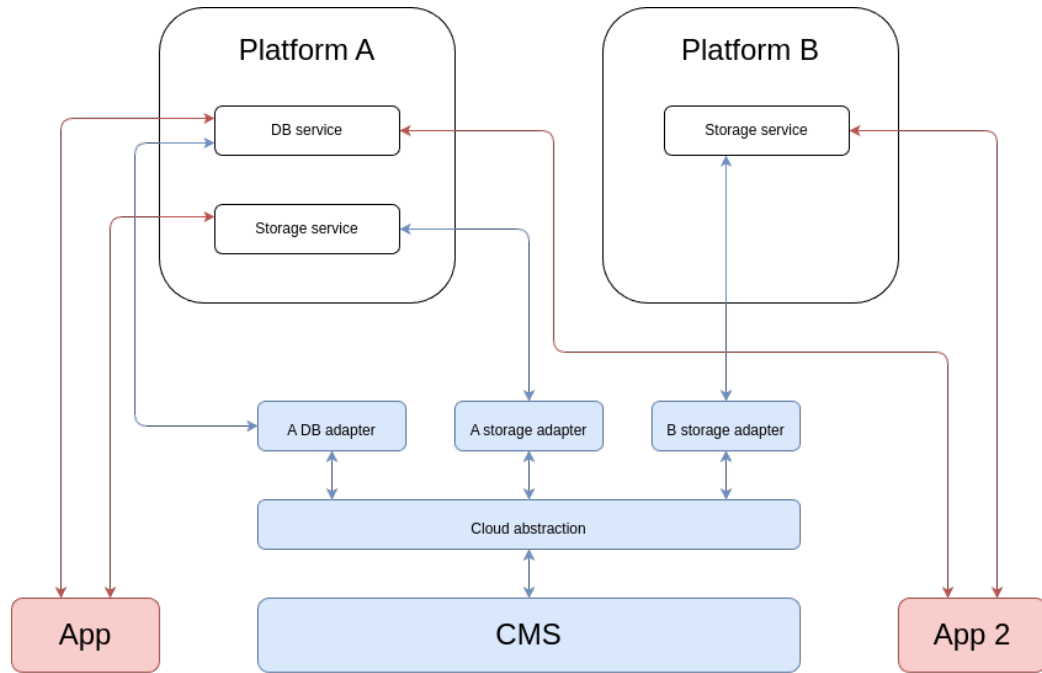


Figure 4.3: General architecture diagram

The whole information system comprises two cloud platforms and three applications, one CMS. The system uses a single database on platform A and two different storage ser-

¹⁵Schemasafe - <https://github.com/ExodusMovement/schemasafe>

¹⁶Ajv - <https://ajv.js.org/>

¹⁷Superforms - <https://superforms.rocks/>

vices on different platforms. This is a fictional system undergoing a migration from one cloud platform to another while also migrating from „App“ to „App 2“. This example visualizes the importance of the above-mentioned flexibility requirements; the CMS manages the database on platform A with a storage service also on platform A and another on platform B. „App“ and „App 2“ include platform-dependent code as it is usually done, the applications are not required to be adapted for use with GenoaCMS. This means adapters are responsible for communication with services; as can be seen, there is an option to have multiple same-service-type adapters per CMS instance. The layer that connects the CMS logic with adapters is called „Cloud abstraction“. In the code, it is represented by the package „@genoacms/cloudabstraction“. This package determines what features must be implemented by adapters and allows CMS logic to utilize them; any breaking change in this package results in the need for a new adapter version and the CMS logic. Finally, there is a block CMS, which represents the CMS logic and its user interface located in the package „@genoacms/core,“ from now on, we will refer to this package as „core.“ The core package is a SvelteKit application whose character is generic and dependent on the configuration file. The core initiates all the adapters and servers the UI for the user. When a user invokes any service-specific action, the server loads related data from the configuration file and calls a particular function from the adapter.

4.4.1 Layered architecture

The system needs to be broken down into layers for a deeper understanding of the architecture. Since the focus is on the internal architecture of the CMS, the following diagram omits usage information.

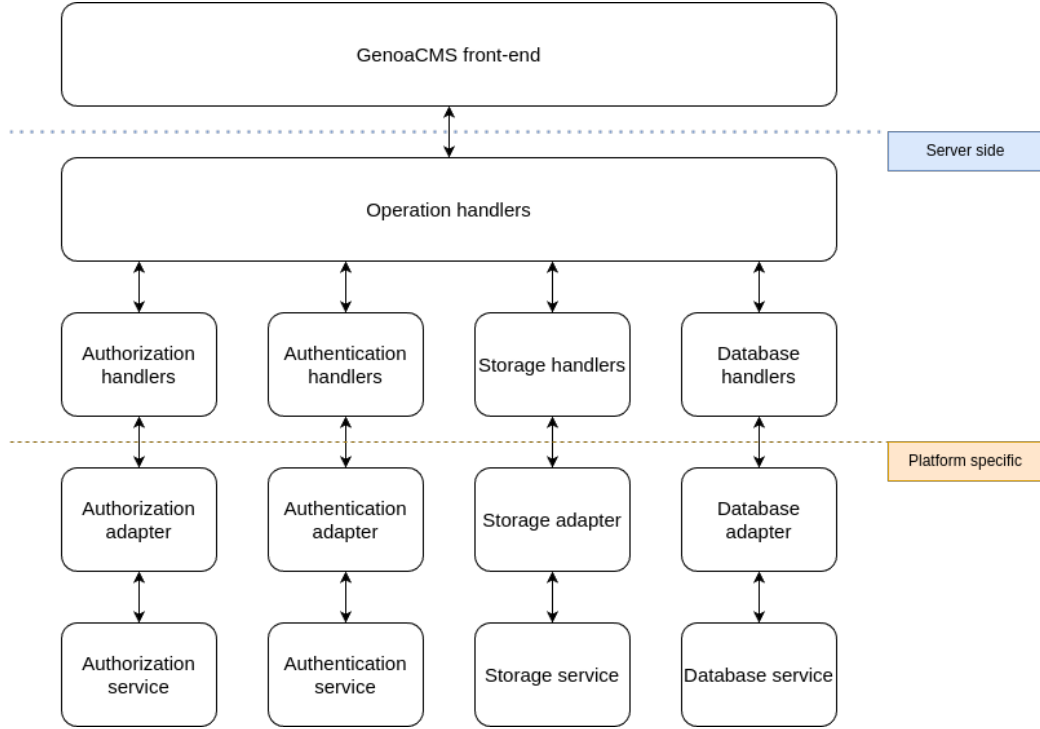


Figure 4.4: Layered architecture diagram

From the bottom, GenoaCMS nomenclature calls this layer the „service layer“. This layer includes actual cloud services and their APIs; in the case of hosting on non-cloud environments, the service layer is represented by the filesystem of a particular server, compute runtime, database server, special files, and others. On this layer can be seen four different services, while on the diagram of general architecture, only 2. That is because the general architecture diagram is simplified to demonstrate usage. GenoaCMS expects at least one adapter from each type of service layer; in exceptional cases, there is an option to run the CMS without either database or storage service, but not without authorization and authentication. This diagram does not mention computing service; the reason is that all the components of the layered architecture run on compute service (either the local machine of the developer or hosted on the server).

A layer above is the „adapter layer,“ which consists of adapter code, translating generic handler function logic to service-specific logic. As can be seen, the adapter code of individual services is separated. However, some operations use a combination of multiple services, and the data aggregations happen exclusively on the operations layer to avoid dependencies between adapters.

The next layer is the „handler layer“, whose only role is to use a correct adapter to a particular resource. Given that GenoaCMS allows working without multiple services, we must differentiate between the execution of operations; for instance, we have two storage buckets: „public“ on GCP and „private“ on AWS; from the configuration, we can infer

which adapter to use for which bucket. This decision is non-trivial, as in some cases, it handles transitive operations, like getting the adapter by collection name; since there is support for multiple databases, we need to resolve the database name from the collection name, then get the provider name from the database name and finally getting the adapter by provider name. This multi-step resolution simplifies the routing of database/storage service pages. It makes it less error-prone since there cannot be a mismatch of auxiliary data required by lacking an on-demand adapter resolution.

„Operations layer“ includes functions that group multiple or directly use a single handler function to execute comprehensive action. A simple example is the function „listOrCreateDirectory“; the CMS uses this function internally to list definitions of components, pages, and other files. Since there is a possibility when listing a directory that it does not exist and having an empty directory is a valid state, it is convenient to have functions handling both cases. For this example, it is important to note that some cloud storage solutions do not have a concept of a directory; instead, they have a concept of an object with its prefix (path); in such cases, the function is implemented by listing all objects and filtering them by prefix. Considering this fact and using prefixes as separators for specific data results in the need to group those operations. Another approach would be storing the data in one prefix and using an auxiliary file/database to store the data associations. However, this approach also requires grouping of operations, and it is less efficient.

The top „front-end layer“ is the Svelte app, which serves the UI to the user and communicates with the operations layer. The front-end layer renders the UI, handles user interactions, and sends requests to the operations layer. Arguably, this layer contains some portion of logic, namely constructors of components and pages; although this logic is responsible for one of the core features of GenoaCMS, it does not break the layered architecture; instead, it builds on top of it. For a detailed explanation of the front-end layer, see section 4.7.

Compute service and deployment are represented as services by this diagram, although it still provides valuable information about its architecture. Notice the „Platform specific“ divider; everything above it is platform agnostic, and everything below it is platform specific. The platform-agnostic part is shipped as it is and cannot be modified by the user, unlike the platform-agnostic part, whose logic can be modified using a configuration file by a user. By using SvelteKit, there is a build step, and since the SvelteKit code begins from the handler layer and continues above, the bundle created by the build groups all platform-agnostic layers. Thus, to run the CMS, that bundle must be paired with the adapter layer code; a generic deployment procedure and deployment adapter do all this. Anything related to deployment is used outside of CMS runtime, which is why the deployment adapter and compute service are missing from this diagram. More about deployment can be found in section 4.8.1.

4.4.2 Package structure

Since there are numerous relations and dependencies between parts of the system, which eventually will be required by adapters or other packages extending the CMS, this led to the decision to decouple packages of GenoaCMS. By isolating functionalities into independent modules, systems can avoid storing and processing unnecessary or redundant data and increase maintainability[10].

GenoaCMS is composed of these packages:

- **@genoacms/core** - contains the front-end layer of the CMS and the operations layer.
- **@genoacms/cloudabstraction** - Typescript type declarations defining the required shape of adapters and config and auxiliary types used within them.
- **@genoacms/cli** - command line utility for initializing, running, and deploying GenoaCMS.
- **@genoacms/adapters-gcp** - reference adapter suite for Google Cloud Platform services.
- **@genoacms/adapters-aws** - reference adapter suite for Amazon Web Services.
- **@genoacms/adapters-postgres** - database adapter for PostgreSQL.
- **@genoacms/adapters-minio** - storage adapter for MinIO¹⁸.
- **@genoacms/adapters-node** - deployment adapter for running from single machine.

The only decoupled packages were „core“ and „cloudabstraction“, giving the ability to import types from „cloudabstraction“ in adapters. The „cli“ package has to be separate since it is an NPM script with assumed usage by calling „npx“, which internally downloads the package. Coupling all the packages would result in downloading the whole CMS twice when attempting to use the CLI and later when initializing the CMS.

GenoaCMS project is set up by calling „npx @genoacms/cli init“ in an empty or node package directory. The script will install all necessary packages and create a configuration start point. Also, the script prompts for the target environment; based on that, it installs the correct adapter suite. If no known adapter exists for the environment, the user has to find it on their own and install it manually.

4.5 Configuration

The shape of the configuration file is defined by cloud abstraction, and the core package expects it to match it (assuming the core and cloud abstraction packages share the same version). This config file is expected to be located in the user's project root in the directory „genoacms.config“ in the file „index.js.“ The configuration file is a JavaScript file and has advantages over static formats like JSON; consider the following:

- **Code reuse** - using a programming language for configuration instead of static format, variables, and functions can shorten the file and make it more readable.
- **Module system** - JavaScript has a module system that allows for importing and exporting data; this is useful for splitting the configuration file into smaller configuration files specialized for certain uses, like separation of service configurations.
- **Type safety** - since GenoaCMS offers type annotations by package „@genoacms/cloudabstraction“, the configuration file can be type checked using tools like jsdoc¹⁹ or even Typescript and compiling it to JS, which is not possible with static formats.

¹⁸MinIO - <https://min.io/>

¹⁹jsdoc - <https://jsdoc.app/>

- **Dynamic values** - JavaScript allows for dynamic values, which can be useful for defining the configuration based on environment variables or for defining the configuration based on other configuration values. This makes possible the separation of environments.

One of the requirements is for the configuration to be serializable. Contrary to specific existing solutions (e.g., Strapi), which have some portion of their configuration as state (thus not reconstructable), GenoaCMS requires every part of its configuration to be in serialized form. On every execution, GenoaCMS reconstructs the in-memory configuration based on the code from a configuration file. GenoaCMS has two types of configuration - static and dynamic. Static configuration is immutable and manages critical system parts, such as providers^{4.5.1}. On the other hand, the dynamic configuration is composed of multiple JSON files on the primary bucket in the directory (prefix) „genoacms.“ Considering current requirements, the dynamic configuration is suitable only for managing buckets and collections.

Consider this type declaration of configuration file:

```
type Config<Extension extends object = object> = Extension & {
  authorization: {
    providers: AuthorizationProvider[]
  }
  authentication: {
    providers: AuthenticationProvider[]
    cookieName: string
    JWTSecret: string
  }
  database: {
    databases: DatabaseInit[]
    providers: DatabaseProvider[]
  }
  deployment: {
    adapterPath: string
  }
  storage: {
    defaultBucket: string
    buckets: BucketInit[]
    providers: StorageProvider[]
  }
  [key: string]: any
}
```

Figure 4.5: Type declaration of configuration

This type declaration shows a clear pattern; it is divided by services, and each service has its providers except „deployment,“ which configures the deployment procedure rather than the service. As can be seen, the configuration of services except deployment includes expected value as an array of providers for their particular type and other configurable properties.

- **cookieName** - sets the name of the cookie used for authentication, as some cloud platforms have restrictions on cookie names.
- **JWTSecret** - secret used for signing JWT authentication tokens.
- **databases** - an array of database initialization objects; each object contains the name of the database, provider name, and collection initializations.
- **defaultBucket** - string specifying which bucket of use for internal data of GenoaCMS.
- **buckets** - an array of bucket initialization objects, each object contains name and provider name.

4.5.1 Providers API

Providers API is an interface enabling the CMS to differentiate between communication with multiple services of the same type. Providers wrap adapters and their config for particular cloud service instances. Adapters are only collections of functions handling communication with specific services; however, the client must be authenticated and appropriately configured to communicate with a service. By grouping adapters with service configuration, it is possible to use the same adapter for the same service but with different configurations for two different projects, e.g., using one adapter for AWS S3 to communicate with two instances of S3 service, one on project A and one on B. Configs for adapters are data required and described by particular adapter. By creating these multiple providers, we must separate resources of the same type (buckets, databases) to avoid conflicts resulting in failed resource resolution. This creates a need for identification of the provider, which is used to resolve the provider for the resource; in this listing [4.6](#), that identification is specified when initializing a provider (property name) and when referring to it in the initialization array of buckets (property providerName). This linking providers with resources only applies to storage and database services.

For illustration, consider the following example of a configuration for storage for two different GCP projects:

```
import credentialsOne from './serviceAccount1.json'
  with { type: 'json' }
import credentialsTwo from './serviceAccount2.json'
  with { type: 'json' }

const cloudStorageProviderOne = {
  name: 'providerOne',
  adapter: import('@genoacms/adapter-gcp/storage'),
  projectId: 'demoOne',
  credentials: credentialsOne
}
const cloudStorageProviderTwo = {
  name: 'providerTwo',
  adapter: import('@genoacms/adapter-gcp/storage'),
  projectId: 'demoTwo',
  credentials: credentialsTwo
}
const config = {
  ...
  storage: {
    defaultBucket: 'private',
    buckets: [
      {
        name: 'private',
        providerName: 'providerOne'
      },
      {
        name: 'public',
        providerName: 'providerTwo'
      }
    ],
    providers: [
      cloudStorage
    ]
  },
  ...
}
```

Figure 4.6: Example of storage config with multiple providers

This config file is located in a well-known place, from whose package's „cloudabstraction“ method „getConfig“ reads. This method is used by both the „core“ and adapters packages. The „core“ package uses it to initialize adapters and render pages based on registered assets (databases, collections, buckets). Adapters use it to resolve their configuration and during their initialization. There is an explicit circular dependency between adapters and config. In

order to resolve this loop, the adapters are imported dynamically without awaiting them, so the resolution starts the first time the config file is run. After adapters are resolved, their code is executed, and they start to initialize; now, they reread the config, but they ignore imports of the adapter modules. While this is happening, any CMS operation awaits the initialization of the adapters, and when they are initialized, the operation is executed. After this, the modules are already loaded into memory, and the next time the operation is called, the adapters are already initialized, and the operation is executed immediately. Although circular dependencies are considered bad practice, the benefits of having a single configuration file outweigh the cons, and specifically, a single configuration entry point was an initial requirement of GenoaCMS.

4.5.2 Authentication

The authentication config includes information about checking the user's identity and transferring that data between the client and server. To have authentication operational, the user must provide at least one provider and set the cookie name and JWT secret. The cookie name is required because some cloud platforms have restrictions on cookie names, and the JWT secret is required to sign JWT tokens. Since there are no resources to link with providers, the requirement for the provider is only the adapter and its config.

4.5.3 Authorization

Similar to authentication, there are no resources to link with providers; additionally, the authorization has no configuration other than providers. The providers are configured the same way as for authentication, and the only requirement is to have at least one provider.

4.5.4 Database

Configuring databases means initializing providers and databases and declaring which database is the primary one. Initializing database providers is analogous to other providers; the only difference is that the database provider requires the database name. The database initialization object includes the name of the database, the provider name, and the list of collections belonging to the database. The primary database is set by string literal, which is the name of the database.

4.5.5 Deployment

By design of deployment^{4.8.1}, the deployment configuration requires an adapter that customizes the deployment procedure to be tailored to a specific compute service. The deployment adapter preserves the same pattern as others and is optionally configurable. GenoaCMS bootstraps the deployment process using its CLI tool with some generic logic. However, there are variations for different compute services APIs and their environments. The deployment process is different from service to service, and the deployed code can vary. Another problem is that just like GenoaCMS, SvelteKit uses adapters primarily for adapting servers. This results in the deployment adapter having to contain a suitable SvelteKit adapter used by the CLI tool during SvelteKit's build step.

4.5.6 Storage

Storage configuration is similar to database configuration, but instead of registering databases, the user registers buckets and declares the default bucket, which is used for string internal data of the CMS (components, pages runtime config). A default bucket can contain sensitive information, and its data is not encrypted. Consequently, it is recommended that this bucket is only for authenticated users. The bucket initialization object includes the bucket's name and the provider's name. The provider name is used to resolve the provider for the bucket.

4.6 Server side

Architecturally, every layer below and, including operation handlers, is located on the server side^{4.4}. GenoaCMS uses server-side rendering to render a page; it first must have all data necessary for it; this is accomplished by SvelteKit's load functions²⁰ which use operation handlers. Operation handlers are functions called by form actions²¹, they are defined in TypeScript files under lib directory²², from where SvelteKit server scripts import them. All data validation happens in the form action functions, and then valid data is passed to operation handler calls.

²⁰Load functions - <https://svelte.dev/docs/kit/@sveltejs-kit#Load>

²¹Form actions - <https://svelte.dev/docs/kit/form-actions>

²²\$lib - [https://svelte.dev/docs/kit/\\$lib](https://svelte.dev/docs/kit/$lib)

Operation handler functions are:

Function	Description
authenticateAndAuthorize	Checks whether the user is authenticated and authorized to use the CMS.
login	Runs checks and sets an authentication cookie.
verifyAuthCookie	Checks authentication cookie.
logout	Deletes authentication cookie.
getCollectionReferences	Lists all collections registered in the configs.
getCollectionReference	Translates collection name to CollectionReference type.
createDocument	Adapter function routed by providers API.
getCollection	Adapter function routed by providers API.
getDocument	Adapter function routed by providers API.
updateDocument	Adapter function routed by providers API.
deleteDocument	Adapter function routed by providers API.
getBucketReferences	Lists all buckets registered in the configs.
createDirectory	Adapter function routed by providers API.
deleteObject	Adapter function routed by providers API.
getObject	Adapter function routed by providers API.
getPublicURL	Adapter function routed by providers API.
listDirectory	Adapter function routed by providers API.
uploadObject	Adapter function routed by providers API.
isDirectoryExisting	Checks whether storage prefix exists.
listOrCreateDirectory	Lists storage prefix or creates it if it does not exist.
getObjectJSON	Reads object with JSON parsing.
getObjectFlatted	Reads object with flatted parsing.
processDirectoryContents	Transforms list of objects into a list of objects with meta-data.

Figure 4.7: Overview of CMS API Functions

Through load functions and form actions, these functions act as an API for the front-end. These functions must be located on the back-end, as they require access to the configuration file and use admin privileges for individual services; they are not intended to be called by the client directly.

4.6.1 Authentication and authorization

All of the functionality is locked behind protected route group²³, which checks every incoming request using function „verifyAuthCookie“ in server handle hook²⁴, unauthenticated users are redirected to login page. The authentication cookie contains a signed JWT. JWT is vulnerable to session hijacking²⁵, to prevent it, the tokens are short-lived. To avoid creating overlapping tokens for different GenoaCMS instances, an option was introduced to configure the JWT secret.

The function **authenticateAndAuthorize** calls the adapter’s functions for authentication and authorization; in the latest authorization specification, there is only one function

²³Route group - [https://svelte.dev/docs/kit/advanced-routing#Advanced-layouts-\(group\)](https://svelte.dev/docs/kit/advanced-routing#Advanced-layouts-(group))

²⁴Handle hook - https://en.wikipedia.org/wiki/Session_hijacking

²⁵Session hijacking - https://en.wikipedia.org/wiki/Session_hijacking

proposed for checking whether the user can or cannot use the CMS; there is no granular control over individual operations. The purpose of the authorization adapter is to serve as a placeholder for future, more advanced authorization systems.

4.6.2 Routing

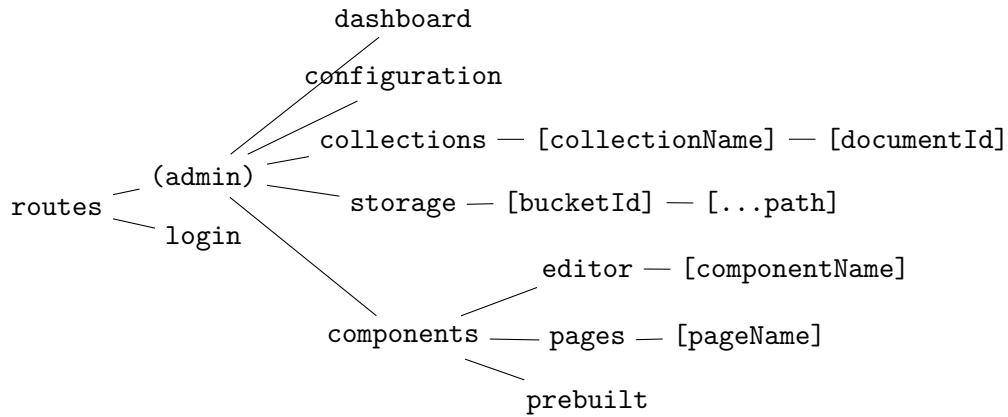


Figure 4.8: Route structure

The routes are structured in service differentiation respecting way with the addition of other routes. To access the CMS, the user must use the login page to authenticate; every unauthenticated request is redirected to this page. The page users are redirected to after login is the dashboard, which acts like a hub.

The configuration page reads and manipulates the runtime configuration, which uses a storage service to access the configuration file.

A route called „collections“ for managing databases lists all registered collections from all databases. Its child route takes parameter²⁶ of collection name, based on it is the page rendered with a list of documents in the collection, this page contains actions for creating new documents. Listing and providing actions for individual documents is done by the child route of the collection route, which takes the document ID as a parameter.

The storage route is responsible for listing all registered buckets without any actions, like collections; adding and removing buckets is exclusively done in configuration. The page for listing and manipulating objects in the bucket is a child route of the bucket route; it takes bucket ID and path as parameters.

Components API includes all routes below the „components“ route. The main shift between components and other managers is that components are represented by auxiliary structures, which must be constructed on the front-end; this results in transferring JSON data to the back-end and more complex data validation. All the data of components API are stored on the primary bucket of GenoaCMS. The editor route is dedicated to listing and creating custom components, and the interface for editing is located in the child route. Its only actions are saving and deleting the component. The page route's role is to list and create pages composed of registered components in the system; its one action is creating a page with a name and root component. Editing the page is done by child route, which takes the page name to resolve the requested page, this page has multiple actions

²⁶Rest parameters - <https://svelte.dev/docs/kit/advanced-routing#Rest-parameters>

sharing common patterns since there are enforced delta uptades²⁷, all action handlers share a familiar pattern, obtaining delta data, validating it, reading original data, merging them and saving the result. Since there is an involved step reading original data, the saved data transfers are only relocated between the compute service and the storage service; however, it is still beneficial, as there is a high probably that both services are running on data centers with high-speed connections in some cases, the communication may be happening on one data center. The next page is called „prebuilt“; it provides an interface for CRUD operations on component representations of those that are already written in the user’s applications. The route has two actions: saving the component and deleting it; there are no delta updates since the assumed size of a component does not require such measures.

4.7 Client side

Client-side structure inherits structure from the server side, the pages are rendered on the server, and operations are called by form submissions, which trigger form actions.

4.7.1 User interface

Although GenoaCMS does not focus on visuals, providing a reasonable user interface and experience is still important. The system separates managing individual services, and the separation applies to the user interface as well.

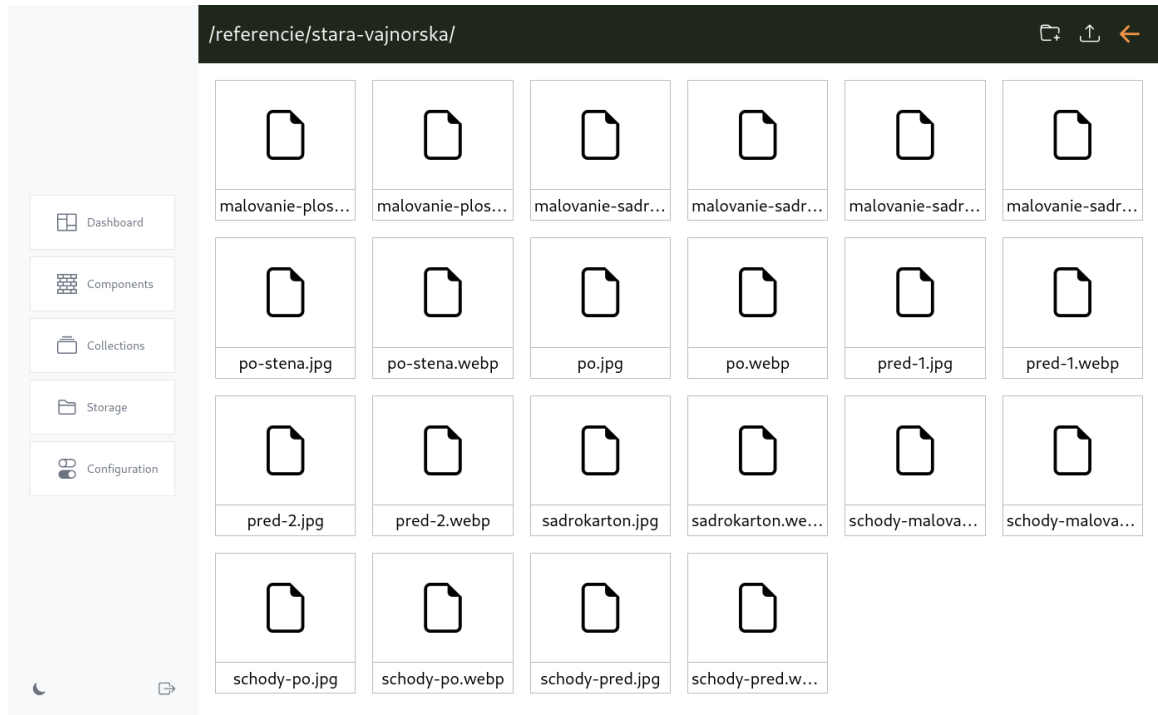


Figure 4.9: User interface screenshot

As can be seen in this figure, the UI is divided into two parts: on the left is the navigation panel, which lists the management of various services, and on the right side is content for

²⁷Delta updates - using partial updates to reduce data transfer

the current service. Also, the navigation panel is used to toggle dark mode and log out. The storage manager can be seen on the right side of the screen. The top dark panel includes currently listed storage prefixes and includes buttons for creating new prefixes, uploading files, or navigating one prefix up; in case of having no prefix, the back button navigates to a list of buckets. Context actions shall be used to manipulate individual objects (getting the access URL, deleting, and other actions). The rest of the service managers obey the same rules; on the top is a dark panel for generally available operations and in-service navigation, and mechanisms like forms, context menus, and modal windows are used for manipulating individual resources.

4.7.2 Component API

Unlike other service managers, the logic of component API lies mainly on the front-end. Also, it deflects from other services by its philosophy, managing database and storage aims for no data dependency on GenoaCMS, meaning the data are the same whether GenoaCMS is used or not. Component API has a format that applications using it must obey.

The special format represents these types:

- **ComponentEntry** - structure describing existing front-end component.
- **ComponentDefinition** - structure defining component by code.
- **Component** - structure combining ComponentEntry and ComponentDefinition.
- **Page** - structure representing page composed out of components.

Note that ComponentDefinition and Component structures are for coding components directly in GenoaCMS, features that are not part of the requirements. Although this feature is not required, it is still part of a proposal to show the potential for future development.

ComponentEntry

This structure contains information about a component's shape, including all of its attributes, types, and limitations. Inherently, when having a component represented by code, there is an expectation that the code will change. Thus, any structure describing the component within the CMS must have the ability to be changed. It is a common practice that changes are reverted, so the component entry features a history of changes.

```

type AttributeReference = string
type ComponentEntryReference = string

interface AttributeBase {
  uid: AttributeReference,
  name: string,
  description: string,
  isRequired: boolean
}

type ComponentEntryAttributes = Record<AttributeReference, Attribute>

type AttributesChange = Array<Diff<ComponentEntryAttributes>>

interface ComponentEntry {
  uid: ComponentEntryReference,
  name: string,
  attributes: ComponentEntryAttributes,
  history: Array<AttributesChange>,
  future: Array<AttributesChange>
}

```

Figure 4.10: ComponentEntry structure proposal

As seen on the listing, the core of the „ComponentEntry“ structure is its property „attributes,“ containing a record of various types of attributes and references to a particular one. The definition of attributes was omitted from the figure, as nine types of attributes are inheriting common properties from AttributeBase with minor differences. Journal of changes will be implemented using two arrays with partial changes of ComponentEntryAttributes defined by type AttributesChange. Finally, there are properties „uid“ and „name“; the name should match the name of the component in the code, so when the user’s application renders the page defined in GenoaCMS, the app can match which component to use for the particular node of a page tree [4.7.2](#). Assuming the Component name can change, we still need identification, which means we do not lose the binding when the users refactor their code; for that, there is uid in ComponentEntry.

When the user builds pages from components, ComponentEntry is linked to the page tree, and based on it, the page editor will create data inputs based on the attributes of the ComponentEntry.

ComponentDefinition

For components written in GenoaCMS, we need to store the code and additional information; for that, there is a ComponentDefinition structure.

```

type ComponentDefinitionReference = string
type ComponentCode = string
type ComponentLanguage = 'javascript'

type CodeChange = Array<Diff<ComponentCode>>

interface ComponentDefinition {
  uid: ComponentDefinitionReference,
  language: ComponentLanguage,
  uncommittedCode: ComponentCode,
  code: ComponentCode,
  history: Array<CodeChange>,
  future: Array<CodeChange>
}

```

Figure 4.11: ComponentDefinition structure proposal

This structure is similar to component entry, although it contains the component’s code instead of attributes. Similarly to ComponentEntry, ComponentDefinition has a history of changes, but in this case, the changes are partial changes of the code. To prevent loss of data, the user’s changes are auto-saved on every input; without separating the user’s intended changes and those caused by auto-save, the user would lose the ability to track them. This led to introducing property „uncommittedCode“ and operation „commit“ to move the value to property „code“ and saving the operation’s diff to „history.“ The code is stored as a string, and until a new proposal, only JavaScript is supported. By supporting only JavaScript, we simplify code analysis but limit this feature to web applications only.

Component

For components written in GenoaCMS, we need to store information about the component’s definition and shape. Because the page manager relies on data from ComponentEntry to render the data inputs, this structure was designed to combine ComponentEntry and ComponentDefinition. This structure is created alongside ComponentEntry and ComponentDefinition when a user creates a component written in GenoaCMS—every change of attributes in code results in a change in ComponentEntry.

```

interface Component {
  uid: ComponentReference,
  name: string
}

```

Figure 4.12: Component structure proposal

Component acts as a declaration that there are existing ComponentEntry and ComponentDefinition structures for the component under a particular name. The uid of the Component is used to bind the ComponentEntry and ComponentDefinition by having the same value across all structures. Decoupling this structure from ComponentEntry and ComponentDefinition is to save data transfers and storage reads/writes. When the user

lists all components, he only needs to know the name and ID of the component. When users access the code, only the `ComponentDefinition` is read; reading `ComponentEntry` has no importance or contrary when using the component within pages; we only need to know the shape of the component, so we only read `ComponentEntry`. Finally, when building the page, we need to access the shape and code of the component so that all the structures are read.

Page

Pages allow nesting components and creating complex structures. Every page has its root component, which determines the shape of the page. Since components allow for subcomponents, the page structure is recursive. Initially, the page structure was designed to be a tree, but due to reasons mentioned in section 5.1.1, the tree structure was replaced by a flat structure with parent-child relations. Page manager offers an interface for creating, editing, and deleting pages; when editing, the user can edit only one component instance at a time, where every change is a delta update.

```

interface AttributeData<T extends AttributeType = AttributeType,
    V extends AttributeValue<T> = AttributeValue<T>> {
    uid: AttributeReference,
    name: string,
    type: T,
    schema: JSONSchemaType<V>,
    value: V
}

type ComponentNodeData = Record<AttributeReference, AttributeData>

interface ComponentNode {
    uid: ComponentNodeReference,
    entryReference: ComponentEntryReference,
    name: string,
    data: ComponentNodeData
}

type ComponentNodes = Record<ComponentNodeReference, ComponentNode>

interface PageContents {
    nodes: Record<ComponentNodeReference, ComponentNode>,
    rootNodeUid: ComponentNodeReference
}

type ContentsChange = Array<Diff<PageContents>>

interface PageEntry {
    name: string,
    previewURL: string,
    contents: PageContents,
    history: Array<ContentsChange>,
    future: Array<ContentsChange>,
    lastModified: string
}

```

Figure 4.13: Simplified PageEntry structure proposal

Similarly to previous structures from components API, the PageEntry features history is based on diffs of the contents. Adding a component to the page creates a new node referencing ComponentEntry; this new node is known as a component instance. ComponentNode binds the ComponentEntry with its data, which is stored in ComponentNodeData. ComponentNodeData binds attribute from ComponentEntry with its value. These relations are used for data validation.

This form of representation of pages is well suited for editing the data but not for rendering the page. The page data has a recursive character. However, the structure is flat, ultimately resulting in the need to parse the data into a tree structure. Since one of the goals is to allow the use of GenoaCMS with any front-end framework, the data must

be accessible without any complexity requiring a front-end library. For this reason, there is a step that transforms `PageEntry` into an easily renderable structure. This step happens on the server side when a user publishes the page.

```
type ReadableAttributeValue = boolean | number | string
  | Array<ReadablePageNode>

type ReadablePageNode<isComponentPrebuilt extends boolean> = {
  componentName: isComponentPrebuilt extends true ? string : undefined,
  componentCode: isComponentPrebuilt extends false ? string : undefined,
  data: Record<string, ReadableAttributeValue>
}
```

Figure 4.14: `ReadablePageNode` structure proposal

`ReadablePageNode` is the tree structure created by parsing `PageEntry`. The structure is recursive, where every node can have children, which are also `ReadablePageNode`. Based on whether the current node is already defined in the user's app, the structure has the name of the existing component or the code of the component written in GenoaCMS. It is stored in JSON format in the primary bucket of GenoaCMS. This is the only structure used by user applications from the components API.

4.8 CLI utility

One of the requirements for creating this CMS was for it to be easy to set up and use. The problem with this kind of system is the deep integration with the user's infrastructure and multi-step setup. By the design of GenoaCMS, the problem of deep integration is resolved, but the multi-step setup remains. To address this problem, a CLI utility was created, which is responsible for initializing, running, and deploying GenoaCMS. The CLI utility is a Node.js script; it will use `clack`²⁸ to interact with the user step by step. As mentioned earlier, GenoaCMS is a set of packages installed in a directory, which means its code is located in „`node_modules`“ of the directory. This creates challenges in interaction with the code, like running or building it - the user must be navigated to the package's directory („`node_modules/@genoacms/core`“) to run or build the code. Also, when deploying the CMS, many minor procedures need to happen. Forcing users to follow those steps manually is more error-prone and time-consuming than having a streamlined approach, like a user interface. Additionally to the deployment, the utility initializes the CMS by creating a new npm project if it does not exist in the current directory, installing the core package, prompting the user for the target platform, installing the correct adapters, and creating a configuration file. The utility also offers an option to run the CMS, which is done by running the `SvelteKit` application in the core package.

²⁸`clack` - <https://www.clack.cc/>

4.8.1 Deployment

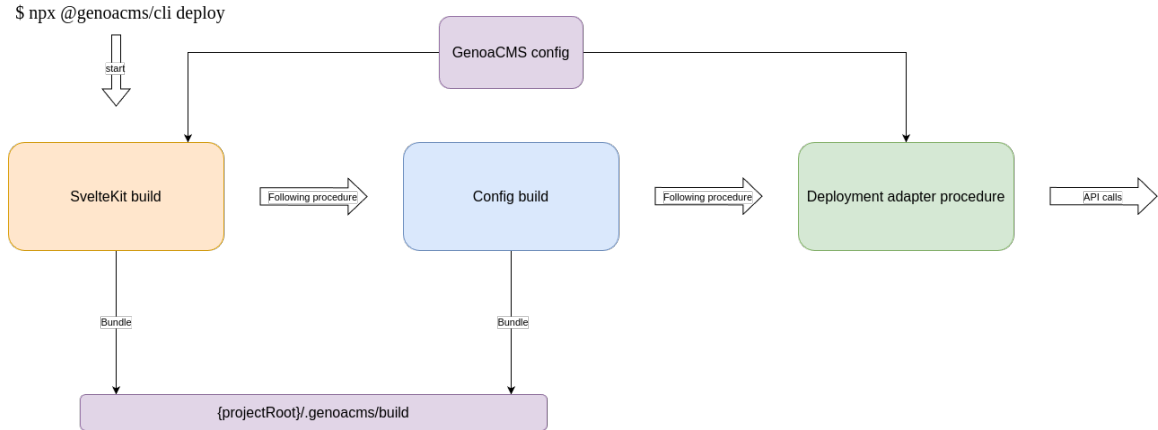


Figure 4.15: Deployment visualization

The process of deployment is composed of these steps:

1. **Building the CMS** - SvelteKit has its build step, which creates a bundle of the application; this bundle varies by used deployment adapter which specifies which SvelteKit adapter²⁹ to use.
2. **Compiling configuration file** - given that the configuration file is in JavaScript and users can modify its behavior by libraries and some platforms have limited sizes for uploaded code, the configuration file must be compiled to JavaScript and minified, rollup³⁰ is used for this. The configuration file imports the adapter modules; they are processed as well.
3. **Running custom adapter logic** - after having runnable files, we need to move them to a place where the compute service can run them. This usually involves calling various service APIs to upload files, set permissions, and other tasks. GenoaCMS cannot predict the exact steps, as they are different for each service, and thus, the deployment adapter has this logic, which is called at this step.

²⁹SvelteKit adapter - <https://svelte.dev/docs/kit/adapters>

³⁰Rollup - <https://rollupjs.org/>

4.9 Offline capabilities

GenoaCMS is designed to be used in environments with limited or no internet connection.

Running GenoaCMS offline can be achieved by using:

- Service emulators
- Special adapters

4.9.1 Service emulators

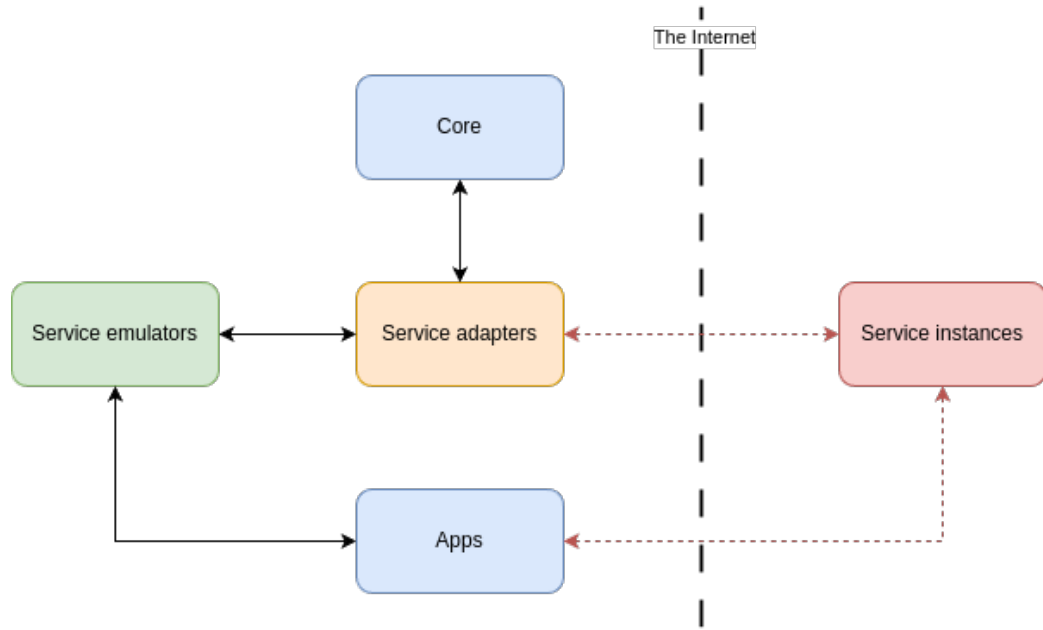


Figure 4.16: Running GenoaCMS offline using service emulators

When the CMS uses certain cloud services, it uses APIs to communicate with remote servers, with those servers communicating with the rest of the system. Some cloud platforms offer emulators, which are programs that allow running services on the user's device while still being able to use original SDKs and libraries; there is only a need to set a flag to use the local emulators. The primary benefit of using emulators is that there is no need to modify the code of applications or CMS configuration. Starting local service emulators is often just a matter of calling one command of a particular cloud service's SDK. Special adapters are the only option to run GenoaCMS offline for non-cloud systems or services without emulators.

4.9.2 Special adapters

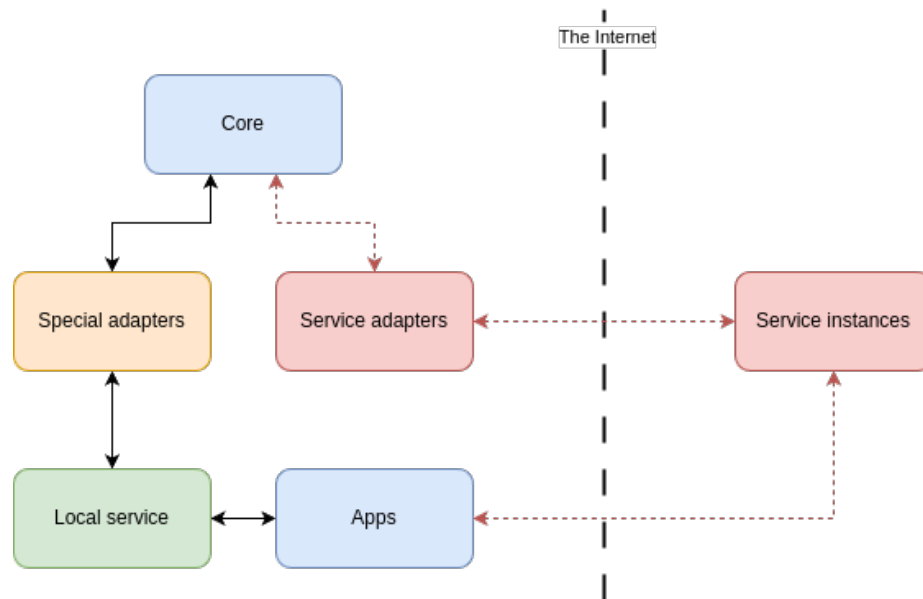


Figure 4.17: Running GenoaCMS offline using special services

Another option to run GenoaCMS entirely on one machine or local network is to use an adapter to communicate with local services. In the case of using GenoaCMS outside of the cloud, this would mean running the local services, reconfiguring the CMS, and using applications. For cloud systems, using special adapters also means reworking the application to use the local service instead of the cloud's APIs, which should be avoided if possible.

Chapter 5

Implementation and testing

Initially, experiments were conducted with GCP and AWS APIs to find the minimal required operations for cloud abstraction. Part of the experiments was building a simple CMS specifically for GCP. Later, when the set of operations was defined, experiments were performed for dynamic import of configuration files with circular dependencies. After confirming the feasibility of the concept, the „cloudabstraction“ package was created, and the prototype of GCP CMS was transformed to be configurable platform agnostic, however, without providers API. Then, the platform-specific functions from GCP CMS were rewritten into an adapter suite for GCP according to „cloudabstraction“. After having a working prototype of platform agnostic CMS configured to use GCP, the effort was put into creating an adapter suite for another platform, AWS. At this point, there were adapters for all services except authentication, authorization, and deployment.

The next step was to implement authentication and authorization. However, this step required experimenting to decide whether to force users to use existing authentication protocols like OAuth and do authentication on the front-end or to implement custom authentication, which offered more flexibility. Experiments showed that using OAuth would create a need to register the CMS as an application on a cloud platform, resulting in two adapters for authentication, one for the front-end and one for the back-end. Custom authentication was shown to be more viable, as it allowed a single adapter for authentication and did not require any other action outside of a config file.

After implementing the custom authentication solution, GenoaCMS lacked a significant part of the requirements - component API. The original proposal included basic principles of the component API. However, it lacked more specific details, like how components and pages will be structured, how the data will be saved, how undo/redo functionality will work, and other problems. To answer these questions, more experiments had to be carried out. However, these required implementing both managing components and pages, as pages use the components. The work started by implementing components according to the initial proposal without any questionable features; next, the very simple prototype of page management was implemented. Both component and page manager tools were implemented as UI for the in-memory monolithic structure. That approach was suitable for the component managers, although an overlooked problem by proposal has emerged when trying to use page manager^{5.1.1}.

Later, undo and redo capabilities for page and component management were implemented. It was done using deep-diff¹ library for comparing changes and storing them in

¹deep-diff - <https://www.npmjs.com/package/deep-diff>

„history“ and „future“ array, yet deep-diff uses references to express changes, and that creates JSON de/serialization challenge, after a brief research, there was chosen to use format and library flattened² for de/serialization for component and page data, except the parsed pages into tree, there is no need for history and in has to be in industry standard format like JSON.

The last lacking feature was the deployment; initially, the focus was on code build and deployment adapters. From the start, the only code that was built was the SvelteKit part of the CMS, but later, when working with environments that do not run pre-run procedure („npm install“), it appeared that having code bundled is necessity^{5.1.3}. The first deployment adapter was for Google Cloud Run, which uploads the code using Google Cloud Functions V2 API, creates or updates the cloud function, and runs it. However, perhaps the prolonged problem was adapting the compute code^{5.1.2}.

After implementing deployment, the CMS was practically usable, so to verify whether GenoaCMS satisfied the requirements, it was implemented into real-world application^{5.2.1}. The real-world verification showed that GenoaCMS was not ready to use, not because of architecture, but rather because of poor implementation of the database manager. However, this result is still a success since it proved that the proposed architecture is vital. The discovered problems were addressed, and the CMS was updated for the real-world use project. At that point, the CMS reached MVP, which meant it was ready to introduce the providers API. Its implementation process proceeded smoothly, with no significant issues arising.

5.1 Major challenges

Often, it showed that the proposal missed some aspects of the system, and cases where the design was not optimal were rare.

5.1.1 Page manager

Unlike the component manager, which defines the shapes of components used on pages, the page manager must be able to access both components and data given to it, which could even be references to storage objects. Since there is a requirement to allow users to source their assets on pages from their storage within GenoaCMS, we must allow users to access their storage from the page manager. However, GenoaCMS already has storage management, but in a separate route as a separate service, and this is a problem; when the managed page is stored in memory until the user is finished with changes, the data cannot be unloaded from memory. One possible solution would be having an embedded storage manager within the page manager, which would mean reimplementing it, even though it is well structured and split into many components and functions. This is the solution of choice for many popular content management systems, despite their significant shortcomings like code duplication, which can ultimately result in unintentional feature disparity between storage managers. Research was done on available alternatives because of the cons of that approach and the fact that it opposed the philosophy of GenoaCMS. Not long after the start of the research, there was an option to use the browser's Broadcast Channel API³ to communicate between page manager and storage manager tab. To try this

²flatted - <https://www.npmjs.com/package/flatted>

³Broadcast Channel API - https://developer.mozilla.org/en-US/docs/Web/API/Broadcast_Channel_API

approach, a prototype of the communication was made. The page manager opens a new tab with the storage manager with a URI flag, signaling the storage manager to initiate a broadcast channel handshake when the page is ready. After the page manager gets a message that the storage manager is ready, it sends metadata about the selection to the storage manager; those data are the number of maximum selectable items and the array of currently selected items. After that, the user can update the selection in the storage manager, and when the selection is confirmed, the storage manager sends the selection back to the page manager and awaits data confirmation from the page manager so the tab can be closed. This approach of storage resource selection was evaluated as successful and integrated into GenoaCMS. Also, the communication protocol was generically implemented and called Inter Tab Communication (ITC), so it can be used for other use cases. This kind of selector returns a storage object selection to one component instance of a page (a structure that nests component instances as a tree), which means that the tree node has to be localized by the ID of the destination component instance of the selection. Component instances have random IDs and are unrelated between the nodes; this means we cannot do a binary search; instead, we can do a Breadth-First Search (BFS) or Depth-First Search (DFS). Consider that the user edits at most one component instance at a time; localizing such a node would have time complexity $O(1)$ in a perfect scenario. So the new question was, how can we avoid BFS or DFS, which typically exhibit $O(V+E)$ time complexity? The simplest option found was to keep only a single component instance in memory; therefore, no search is needed when receiving the data from selection. However, this only moved the problem to the back-end, which had to locate the component instance in the tree and return that to the front-end. The solution to that was breaking the tree into objects with an array of component instances and properties pointing to the root component instance. This resulted in quicker node search with complexity $O(n)$ when loading the page, reduced data transfers, and faster load times. On the other hand, for applications rendering the pages, this creates the need to parse the array to tree structure, and for CMS, having a goal not to require a front-end library to manage parsing for users is a problem. The use of an array instead of a tree was reconsidered, but the pros outweighed the cons, and the solution for the parsing was solved by parsing it by the CMS after the user published the page.

5.1.2 Cloud Run SvelteKit adapter

It is not unusual to have a variety of environments when running web applications on compute services. Some services allow only processing requests. Thus, another service must handle static assets. From the perspective of the architecture of GenoaCMS, even the code handling requests are part of the produced code by the deployment adapter. When writing a deployment adapter for GCP, SvelteKit node adapter⁴ was used, its output is a handler function, server using it and static assets. The server was removed, and a handler function was provided to Google Cloud Functions, which ran the code using Cloud Run, which acts like a server. The deployed code seemingly worked properly, but it ultimately failed on every non-GET request, returning code 400. The first challenge when resolving this problem was isolating it, as that was part of the development of the first deployment adapter, and the problem could be in the adapter or in the platform itself. For a long time, this problem was unresolved, most precisely eight months; during that time, other development tasks were executed. At first, Google's logs showed that the

⁴@sveltejs/adapters-node - <https://www.npmjs.com/package/@sveltejs/adapters-node>

response was returned from „Google Fronted Servers“, and logs I inserted to the server were not displayed, which indicated the error could originate in Google’s infrastructure, like the load balancer. Google’s support for their cloud platform is a paid service with a high price for this self-financed project; because of that, we contacted Google’s engineer on YouTube, although we did not get any valuable results. Finally, the issue was isolated to the SvelteKit handler’s request parsing, namely HTTP headers. Based on this finding, we contacted the SvelteKit community. We got a response from Stanislav Khromov⁵ and he indeed confirmed the problem was in SvelteKit’s node adapter and writing one for Cloud Run would be necessary. He recommended inspiring with an existing serverless adapter; he explicitly mentioned Cloudflare’s adapter. There was found adapter for Firebase functions⁶, which was similar to Cloud Run, the base for the new adapter was used the node adapter with parsing of HTTP headers inspired by Firebase adapter. The new SvelteKit adapter fixed the 400 error on non-GET requests. This experience highlights the importance of deep knowledge of the compute platform when writing deployment adapters, which could be considered a drawback of GenoaCMS.

5.1.3 Build

Some computing services, like Google Cloud Functions, run post-deploy procedures to install and build the deployed code. On the other hand, services like AWS Lambda do not have that step. Thus, we cannot make that step requirement if GenoaCMS should be platform agnostic. Without installed dependencies on the compute service, deployment adapters must upload the whole code to the service, including the „node_modules“ directory. This creates a problem when considering the maximum upload archive size, which is often exceeded when including „node_modules“. The solution is bundling the code with features like tree-shaking to strip all unused code. SvelteKit has a built-in bundler. However, it is optimized only for web applications and unsuitable for bundling configuration files. In order to deploy to AWS Lambda, the configuration file with adapters had to be bundled separately. This is one of the rare cases where the original deployment proposal was inaccurate, and the build step for config (platform-specific code) had to be included.

5.1.4 Loading dynamic configuration

Configuring collections (and buckets) dynamically emerged as a feature introducing instability and significant security problems. Dynamic configuration is loaded on-demand from the JSON file in the primary bucket. Experiments were conducted with unencrypted and unsigned configuration files. Usually, confidentiality is not a problem when configuring database collections. However, authenticity is a major problem. When considering the requirement of serializable configuration whose parts could be moved from dynamic configuration to static without any processing, encrypting the file is not an option. Thus, we cannot ensure confidentiality. Implementing configuration signing would be easy; however, because GenoaCMS lacks granular authorization, every user could edit the dynamic configuration. For production systems relying on external contractors to manage data, this could result in contractors accessing the whole database/cloud storage. Solving this problem properly would result in the need for a more sophisticated authorization system in the first place, and the proposal would have to be fundamentally changed. As a result, the

⁵Stanislav Khromov - <https://khromov.se/>

⁶svelte-adapter-firebase - <https://www.npmjs.com/package/svelte-adapter-firebase>

dynamic feature was restricted to a minimal form, where users were not allowed to edit it except by rewriting the files manually. This prevents abuse from less knowledgeable users. However, the CMS is still vulnerable.

5.2 Testing

Testing highly customizable systems such as GenoaCMS cannot be done in one step when developing the CMS, as its behavior vastly depends on used adapters. In this stage of development of GenoaCMS, the system includes only unit tests for adapters and lacks integration and end-to-end tests. Experiments were conducted with real-world applications to substitute for the lack of those tests. There is a plan to implement integration tests and end-to-end tests using Playwright⁷, although, before that, there are scheduled implementation of quality of life improvements.

5.2.1 Real world usage

During the development of GenoaCMS, the author did not work on any projects that would be suitable for testing the CMS, but at the time, the author's presentational websites were outdated; for that reason, it was determined that the first real-world implementation would be for author's presentational websites. For performance and cost-effectiveness comparison, it would be necessary to have one product (website or information system) with GenoaCMS and another with its alternative. However, the purpose of this real-world usage is to prove whether GenoaCMS could be considered as usable. The first use of GenoaCMS was for managing the collection of past projects of one privately owned company, mainly using database and storage management. The company's past project was named „Reference“, which was represented as a structure with a name, description, image, and sections. The sections had to be one of three types and could vary based on the user's choice. At this point, it was revealed that the CMS lacked support for deeply nested objects and arrays, which are crucial for most modern applications; without this feature, the CMS was not usable for the project. After adding this capability, GenoaCMS could be considered as feature complete. However, it lacked various quality-of-life features, like users' ability to do complex queries, drag and drop for arrays in the collection manager, rename objects in the storage manager, and others.

After having verified that the CMS was usable, the next step was verifying whether it was indeed platform agnostic and whether migrating from cloud to cloud was just a drop-in replacement of adapters and credentials. AWS projects were created, and S3 buckets, DynamoDB, and IAM policies were initialized. GCP adapters were replaced for AWS ones except for the deployment; unresolved issues with the AWS Lambda adapter prevented full-fledged usage. The experiment proved that, except for the deployment adapter, the AWS adapter suite is production-ready, and migrating from platform to platform is only a matter of changing the configuration file.

⁷Playwright - <https://playwright.dev/>

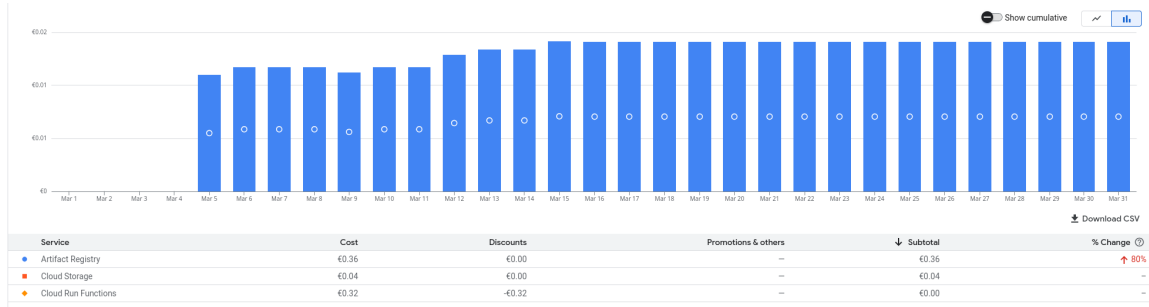


Figure 5.1: Cost analysis of a website using GenoaCMS

One of the real-world websites using GenoaCMS is „fimtrade.com,“ which is purely presentational; most of the time, the CMS is idling. The system contains two containers, a website server and a GenoaCMS server, both scale to 0 instances when unused for more than 10 minutes. During the month in question, GenoaCMS was redeployed often, which can be seen in the dominant cost of artifact registry, which Cloud Run uses for deployment. Only a tiny cost was generated by actual compute time and cloud storage. Thus, it can be seen that GenoaCMS is the optimal solution for small systems that can scale to zero.

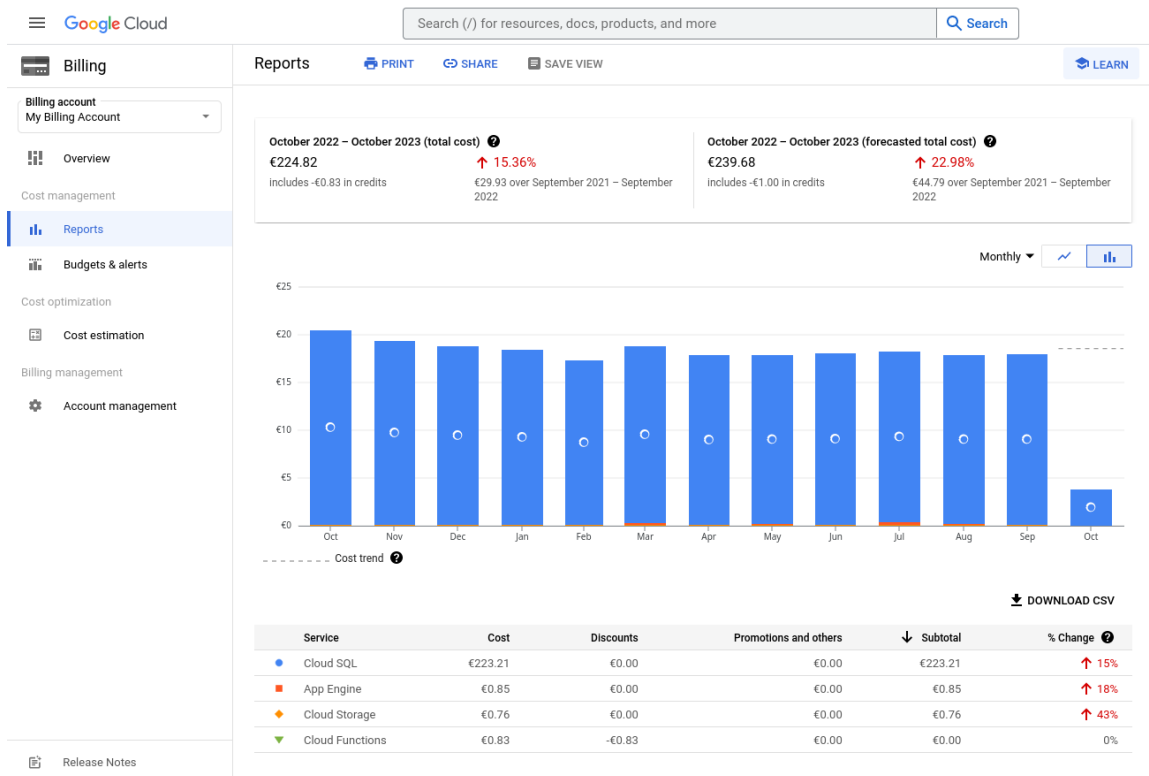


Figure 5.2: Cost analysis of a website using GenoaCMS

A similar website by structure and visits is „insaid.eu“; however, this one uses Strapi 4, which requires an SQL database that is billed even for idle. This report is more precise than the previous one because it covers the annual spending of the website. However, if we multiply the cost of Fimtrade’s website by 12 (8.64€) and compare it to Insaid’s 224.82€,

the difference is vast. It is important to note that Insaid's reports are older and use older cloud functions with different configurations from Fimtrade's. However, it is evident that Cloud SQL was the cost generator.

Chapter 6

Conclusion

The quest to build GenoaCMS was started to prove that tools like content management systems should not shape the architecture of information systems and that CMS solutions can be platform agnostic and run and be developed without connection to the internet. Purely by theory, such a system would be possible, but the practice showed how such a system could work. Building GenoaCMS highlighted the importance of type systems, data scheming, and abstractions for highly flexible systems.

Probably, GenoaCMS will not become mainstream CMS, as that would require a substantial amount of man-hours to keep up feature-wise with other solutions. However, it is a successful proof of concept from which other CMS solutions can draw inspiration. The set of systems that can be inspired from it is not limited to content management systems; during the development of the CMS, experiments were also conducted using the package „cloudabstraction“ with adapters for applications to make them similarly adaptable. Before creating the „cloudabstraction“ package, there was research on existing solutions to make platform-agnostic applications. There were no simple solutions like a library that makes applications adaptable to any environment. There are solutions like Pulumi¹ or Terraform², but their approach is vastly different. Using a bridge pattern to achieve platform agnosticism shows a viable solution, and this approach is expected to be used in future projects despite the pattern being well known.

Still, the development continues, as GenoaCMS proved a simple yet powerful CMS for small to medium-sized projects. There are still unfulfilled ambitions, like the above-mentioned component code editor, end-to-end tests, i18n support, and more.

GenoaCMS is regarded as a successful implementation, as it validated the concept of a platform-agnostic CMS and met all specified requirements. Furthermore, an experiment measuring idle system usage over three months demonstrated that GenoaCMS can remain deployed and accessible at minimal cost, with Google issuing only a symbolic invoice of €0.01.

¹Pulumi - <https://www.pulumi.com/>

²Terraform - <https://www.terraform.io/>

Bibliography

- [1] BARKER, D. *Web Content Management: Systems, Features, and Best Practices*. 1st ed. O'Reilly Media, 2016. ISBN 978-1491908129.
- [2] BLINOWSKI, G.; OJDOWSKA, A. and PRZYBYLEK, A. Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation. *IEEE Access*, january 2022, vol. 10, p. 1–1.
- [3] GAMMA, E.; HELM, R.; JOHNSON, R. and VLISSIDES, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. 1st ed. Addison-Wesley Professional, 1994. ISBN 978-0201633610.
- [4] HARRIS, R. *Yes but does it scale?* online. 02. april 2024. Available at: <https://github.com/sveltejs/svelte/issues/2546>. [cit. 2025-17-03].
- [5] KLAUSMEIER, E. *Performance Comparison C vs. Java vs. Javascript vs. PHP vs. Python vs. Cobol vs. Dart* online. 25. march 2023. Available at: <https://eklausmeier.goip.de/blog/2023/03-25-performance-comparison-c-vs-java-vs-javascript-vs-php-vs-python-vs-cobol-dart>. [cit. 2025-16-01].
- [6] MEYER, B. *Object-oriented software construction*. 2nd ed. Prentice Hall PTR, 1997. ISBN 978-0136291558.
- [7] NARAYAN, A.; KUMAR, A.; SHARMA, V.; SAMI, M.; ASHIKA, A. et al. Decoding the Web CMS Landscape: A Comparative Study of Popular Web Content Management Systems. *International Journal of Computers and Their Applications*, december 2024, vol. 31, p. 281–291.
- [8] NECULA, G. C.; CONDIT, J.; HARREN, M.; MCPPEAK, S. and WEIMER, W. CCured: type-safe retrofitting of legacy software. *ACM Trans. Program. Lang. Syst.* New York, NY, USA: Association for Computing Machinery, 2005, vol. 27, no. 3. ISSN 0164-0925. Available at: <https://doi.org/10.1145/1065887.1065892>.
- [9] STAIR, R. and REYNOLDS, G. *Fundamentals of Business Information Systems*. Cengage Learning, 2008. ISBN 978-1408008843.
- [10] THAIYA, M. S.; JULIA, K. and MBUGUA, S. *On Software Modular Architecture: Concepts, Metrics and Trends* online. 2022. Available at: https://www.researchgate.net/publication/360726289_On_Software_Modular_Architecture_Concepts_Metrics_and_Trends. [cit. 2025-18-02].
- [11] WRIGHT, A.; ANDREWS, H.; HUTTON, B. and DENNIS, G. JSON Schema: Core Specification - 2020-12, 2022. Available at:

<https://json-schema.org/draft/2020-12/json-schema-core.html>. Accessed:
2025-01-23.

Appendix A

The future

As stated in the conclusion of this work, the goal was to design and implement a CMS with previously unseen non-functional features (for a CMS). However, the product is more of a proof of concept than a competitive CMS. This appendix was written after completing the goal and continuing the development while scrutinizing the product in a production environment. During this period, storage operations were enriched by adding adapter-level directory deleting, moving, and renaming objects/directories; for database management, cross-database document references were implemented, and UI for managing documents was reworked. Tracking user activity for convenient navigation within the CMS was implemented. The coding components feature was prototyped and is usable, but with serious drawbacks. Furthermore, the whole codebase was open-sourced¹ and documentation was written. Although these features were not part of the original goal, experience while developing them uncovered more areas for improvement.

A.1 Coding components

The most prominent planned feature is coding components within the CMS. Contrary to regular components, these are registered and defined exclusively in the CMS, and the app rendering the page executes the code with the provided data. Fundamentally, this is remote code execution², and having this feature would result in a set of technical security-related challenges. The current implementation is experimental and lacks any security analysis of the code. Thus, its use is not recommended in a production environment. Another drawback is that the only supported language is TypeScript, which limits the potential usage to web only. Originally, the idea was to use plain JavaScript, but the component must be parametrized; for that, TypeScript was used. The code analyzer expects a function with the name of the component and function parameters with types as special types (defined by GenoaCMS). From types used in function parameters, GenoaCMS infers „ComponentEntry,“ which can be used to build pages. When the app renders the page, it loads the compiled code and runs the function with parameters from the „ComponentNode“ in the shape defined by the „ComponentEntry.“ There are plans to support more front-end platforms and do it securely in the future. Current analysis and compilation are simple and unoptimized; they use the TypeScript compiler API and future solutions should rely less on external tools while supporting more platforms. Developing a custom domain-

¹GenoaCMS organization - <https://github.com/GenoaCMS>

²Remote code execution - https://en.wikipedia.org/wiki/Arbitrary_code_execution

specific language that is compilable to platform-specific code could also be an option. When rendering the page, the developer cannot unquestioningly trust that the retrieved code from the storage is safe. To ensure the code is safe, the page should run the security analysis every time it is rendered; this is another factor to consider when developing this feature. The current version control is a git imitation, with support only for commits and checking out a specific commit; in the future, there could be an option to load a git repository as the component library and manage it from the CMS.

Creating a solution with a combination of these features is a challenging task, perhaps even more time-consuming than reaching the current state of GenoaCMS. However, it could be a significant leap forward in the CMS world.

A.2 Activity tracking

Currently, tracking users' activity is done exclusively on the front end, and it is only for the user's convenience, not for reporting or auditing. We plan to move the data gathering to the back-end and store it alongside internal CMS data. Later, that data could be used for the user's convenience as well as auditing and reporting.

A.3 Artificial intelligence service

At the time of writing, AI is experiencing a wide adoption in the software industry. GenoaCMS has several features that AI could enhance. For example, when managing a document, the CMS could assist the user with writing the content. It could also be used to simulate activity on inactive pages by periodically generating content for the page. AI could assist with code completion for writing components similar to code assist in IDEs. It could rename images in one directory to describe their content when managing the storage. These features would be introduced as another service alongside the current services and would be portable to any AI service provider using adapters.

A.4 Onboarding developers

After introducing the above-mentioned features, GenoaCMS will grow in complexity beyond maintaining the capabilities of a single developer. To ensure the project is maintainable and has a future, the project attracts contributions from other developers. The project's open-source nature will make it possible for this project to survive. However, providing good documentation and promoting the project among developers is necessary.